

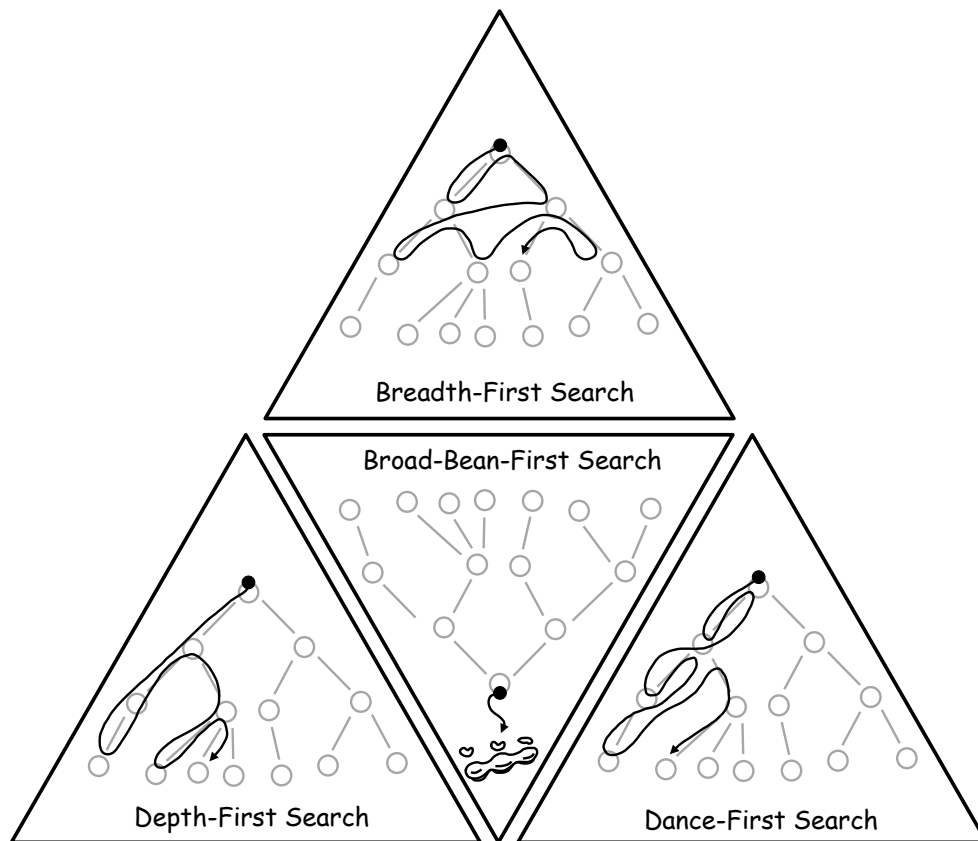
UNIVERSITÀ DEGLI STUDI DI PERUGIA

Dipartimento di Matematica e Informatica

Corso di laurea triennale in INFORMATICA

Algoritmi e Strutture Dati

Esercizi Risolti



F. Coró

L. Palazzetti

S. Bonacorsi

A cura di:
C. M. Pinotti

Tabella dei Contenuti

1	Studio della Complessità	1
1.1	Ordini di Grandezza	1
1.2	Analisi di Codici Iterativi	6
1.3	Analisi delle Equazioni di Ricorrenza	10
1.4	Analisi di Codici Ricorsivi	15
2	Algoritmi di Ordinamento	21
2.1	Ordinamento	21
3	Divide ed Impera	29
3.1	Ricerca di Elementi nelle Matrici	29
3.2	Ricerca di Elementi nei Vettori	31
4	Heaps	39
4.1	Problemi su Max-Heaps	39
5	Statistica d'Ordine	43
5.1	Ricerca di Elementi per Rango	43

Quest'opera è distribuita con licenza Creative Commons “Attribuzione
– Non commerciale 4.0 Internazionale”.



Introduzione

Il seguente libro contiene esercizi sugli argomenti del corso: Algoritmi e Strutture Dati; è considerato da accompagnare allo studio del libro di testo [1], per una corretta preparazione all'esame.

I capitoli rappresentano i macro-argomenti, gli esercizi sono raggruppati per tipologia nei sottocapitoli.

Ogni esercizio ha a seguire lo svolgimento; quelli con svolgimento più dettagliato si trovano all'inizio del capitolo. Quindi procedendo nel capitolo gli svolgimenti degli esercizi diverranno progressivamente più sintetici nelle spiegazioni.

Gli esercizi iniziali dei sottocapitoli tendono ad essere prolissi nelle spiegazioni, lo studente durante un esame deve essere più sintetico rispetto ad essi.

Si prenda quindi come esempio di svolgimento in sede di esame quelli finali dei sottocapitoli, dove la risoluzione è immediata, con solo le spiegazioni necessarie richieste dall'esercizio.

Prima di ogni esercizio sono presenti delle etichette contenenti informazioni più specifiche sulle tecniche o algoritmi utilizzati in esso (Es: #TeoremaEsperto, #Dijkstra, #Iterazione); vedere l'ultima pagina del documento per l'indice analitico delle etichette.

Gli esercizi provengono da esami ed esercitazioni.

Capitolo 1

Studio della Complessità

1.1 Ordini di Grandezza

Esercizio 1.1.1 #OrdineGrandezza Verifica che :

- $\Theta(1) + \Theta(1) \in \Theta(1)$
- $\Theta(1)\Theta(f(n)) \in \Theta(f(n))$

Verifichiamo $\Theta(1) + \Theta(1) \in \Theta(1)$ applicando la definizione di ordine di grandezza. Innanzitutto, ricordiamo che $\Theta(1)$ è l'insieme delle funzioni $f(n)$ per cui esistono due costanti positive c_1 e c_2 che le limitano, rispettivamente, inferiormente e superiormente a partire da un certo valore n' . Possiamo riscrivere il lato sx della proposizione come esistono $a, a', b, b' \geq 0$ per cui $\forall n \geq n'$ si ha: $a + a' \leq 1 + 1 \leq b + b'$. Quindi l'espressione a sx della proposizione è $\Theta(1)$ scegliendo $c_1 = a + a'$, $c_2 = b + b'$, e $n_0 = n'$.

Verifichiamo $\Theta(1)\Theta(f(n)) \in \Theta(f(n))$. Assumiamo che per $\forall n \geq n'$ ove n' soddisfa sia $\Theta(1)$ che $\Theta(f(n))$, $\Theta(1)$ e $\Theta(f(n))$ si possono riscrivere, rispettivamente, come

$$a \leq 1 \leq b$$

e

$$a' \leq f(n) \leq b'$$

per ogni $n \geq n'$. Quindi l'espressione a sx per $n \geq n'$ soddisfa la definizione $\Theta(f(n))$ scegliendo $c_1 = aa'$, $c_2 = bb'$ in quanto $aa'f(n) \leq \Theta(1)\Theta(f(n)) \leq bb'f(n) \forall n \geq n'$.

Esercizio 1.1.2 #OrdineGrandezza

Applicando la definizione di ordine di grandezza, provare o confutare la seguente affermazione: Data $f(n) = 3^{2 \log_3 n + 3}$, $f(n) \in O(n^3)$.

Diamo di seguito diversi modi di risolvere.

Osserviamo che $3^{2 \log_3 n + 3} = 3^{2 \log_3 n} \cdot 3^3 = 27n^2$. Trattandosi di un polinomio di grado 2, $f(n) \in \Theta(n^2)$ che implica $f(n) \in O(n^3)$.

Proviamo ora applicando la definizione di ordine di grandezza:

$$\exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0, 3^{2 \log_3 n + 3} \leq c \cdot n^3$$

$$3^{2 \log_3 n + 3} \leq c \cdot n^3 \rightarrow 3^{2 \log_3 n + 3 \cdot \log_3 3} \leq c \cdot n^3 \rightarrow 3^{\log_3(n^2 \cdot 3^3)} \leq c \cdot n^3 \rightarrow n^2 \cdot 3^3 \leq c \cdot n^3 \rightarrow 3^3 \leq c \cdot n$$

L'ultima disequazione si dimostra vera fissando (ad esempio) $c = 9$ e $n_0 = 3$. Pertanto $3^{2 \log_3 n + 3} \in O(n^3)$.

Un altro modo ancora è notare che:

$$\lim_{n \rightarrow \infty} \frac{n^2 \cdot 3^3}{n^3} = 0$$

e quindi $3^{2 \cdot \log_3 n+3} \in o(n^3)$, da cui si deriva che $f(n) = O(n^3)$ perchè vi sono infinite costanti che verificano la definizione di O .

Ora invece chiediamoci se $3^{2 \cdot \log_3 n+3} \in \Omega(n^3)$. Dal primo metodo di soluzione e dall'ultimo, sappiamo che l'affermazione è falsa. Anche applicando la definizione di Ω , si verifica che è falso. Infatti dovrebbe esistere una costante c per cui per ogni $n \geq n_0$ vale:

$$\exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0, 3^{2 \cdot \log_3 n+3} \geq c \cdot n^3$$

$$3^{2 \cdot \log_3 n+3} \geq c \cdot n^3 \rightarrow 27n^2 \geq c \cdot n^3 \rightarrow c \leq \frac{27}{n}$$

Tale disuguaglianza invece non vale per $n \rightarrow \infty$ per nessuna costante c . Per qualsiasi valore di c , la disuguaglianza vale solo per $n \leq \frac{27}{c}$.

Possiamo quindi concludere che $3^{2 \cdot \log_3 n+3} \notin \Theta(n^3)$, ma $3^{2 \cdot \log_3 n+3} \in O(n^3)$.

Esercizio 1.1.3 #OrdineGrandezza Applicando la definizione di ordine di grandezza, provare o confutare la seguente affermazione.

Data $f(n) = \log_b(n!)$, $f(n) \in \Theta(n \log_b n)$ con $b > 1$.

Per provare o confutare l'appartenenza di $f(n) = \log_b(n!) \in \Theta(n \log n)$ è necessario verificare l'appartenenza a $O(n \log n)$ e contemporaneamente a $\Omega(n \log n)$. Nel seguito della discussione assumiamo $b = 2$, ma è solo per comodità di calcolo. Infatti ci ricordiamo che due logaritmi in due basi diverse sono uno Θ dell'altro.

Ossia: $\log_b n \in \Theta(\log_a n)$ perché

$$\log_b n = \frac{\log_a n}{\log_a b} = \frac{1}{\log_a b} \log_a n.$$

Pertanto cominciamo con il valutare $\log(n!) \in O(n \log n)$.

Ossia, applicando la definizione di O ,

$$\exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0, \log(n!) \leq cn \log n$$

Sappiamo che:

$$n! = n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdot \dots \cdot 2 \cdot 1$$

Possiamo maggiorare lavorando su ogni singolo termine esplicitando per $n \geq 2$:

$$\leq \underbrace{n \cdot n \cdot n \dots n}_{n \text{ volte}} = n^n$$

La funzione logaritmo con base maggiore di 1 è una funzione crescente.

Pertanto poiché $n! \leq n^{n-1} < n^n$ per $n_0 \geq 2$, si ha :

$$\log(n!) < \log(n^n)$$

Quindi:

$$\log(n!) < \log n^n = n \log n$$

Quindi esiste $c = 1$ e $n_0 = 2$, che verificano $\log(n!) \leq cn \log n$.

Concludiamo perciò $\log(n!) \in O(n \log n)$.

Valutiamo ora se $f(n) = \log(n!) \in \Omega(n \log n)$:

$$\exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0, \log(n!) \geq cn \log n$$

Così come fatto per $O(\cdot)$ proviamo a riscrivere il fattoriale in modo da rendere evidente la relazione di appartenenza.

$$n! = n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdot \dots \cdot 2 \cdot 1 \geq \underbrace{n \cdot (n-1) \cdot (n-2) \cdot (n-3) \cdot \dots \cdot \left\lceil \frac{n}{2} \right\rceil}_{\left\lfloor \frac{n}{2} \right\rfloor + 1 \text{ termini}} \geq \frac{n^{\frac{n}{2}}}{2}$$

Pertanto:

$$\log(n!) \geq \log \frac{n^{\frac{n}{2}}}{2}$$

Fissato $n_0 \geq 16$:

$$\log(n!) \geq \frac{n}{2} \log \frac{n}{2} \geq \frac{n}{4} \log n$$

perché $\log_2 \frac{n}{2} \geq \frac{\log_2 n}{2}$.

Dunque: esiste $c = \frac{1}{4}$ e $n_0 = 16$, per cui:

$$\log(n!) \geq cn \log n$$

La tesi è verificata, precisamente:

$$\log(n!) \in \Theta(n \log n)$$

Esercizio 1.1.4 #OrdineGrandezza #ConfrontoFunzioni

Ordinare le seguenti funzioni in ordine di grandezza:

$$n^2; (\sqrt{n})^3; 4^{\log_{16} n}; n^{2/3}$$

$$4^{\log_{16} n} = n^{1/2}$$

$$(\sqrt{n})^3 = n^{3/2}$$

Pertanto, ordinando gli esponenti: $n^2 > (\sqrt{n})^3 > n^{2/3} > 4^{\log_{16} n}$

Esercizio 1.1.5 #OrdineGrandezza #ConfrontoFunzioni

Ordinare le seguenti funzioni in ordine di grandezza:

$$2^{\log_2 \log_2 n}; \sqrt{n}; 4^{\log_2 n}; n^{2/3}$$

Attenzione allo scambio di argomento con la base:

$$2^{\log_2(\log_2 n)} = \log_2 n$$

$$\sqrt{n} = n^{1/2}$$

$$4^{\log_2 n} = n^2$$

Quindi l'ordine è: $4^{\log_2 n} > n^{2/3} > \sqrt{n} > 2^{\log_2 \log_2 n}$

Cioè: $O(\log_2 n) \in O(n^{\frac{1}{2}}) \in O(n^{\frac{2}{3}}) \in O(n^2)$.

Esercizio 1.1.6 #OrdineGrandezza #ConfrontoFunzioni

Siano $f(n)$ e $g(n)$ due funzioni asintotiche positive. Provare o confutare la seguente congettura.

$$f(n) + g(n) = \Theta(\min(f(n), g(n)))$$

Per dimostrarlo è necessario provare che:

$$f(n) + g(n) = \begin{cases} O(\min(f(n), g(n))) \\ \Omega(\min(f(n), g(n))) \end{cases}$$

Cominciamo con Ω :

- Senza perdita di generalità assumiamo che $f(n)$ sia più piccola asintoticamente di $g(n)$.

Abbiamo che $f(n) = \min(g(n), f(n))$, e $g(n) = \max(g(n), f(n))$. Quindi da questo segue che $f(n) + g(n) \geq 2 \min(g(n), f(n))$.

Rimane da verificare O . Per questo mostriamo con un controesempio che non è vero.

- Sia $f(n) = n$ e che $g(n) = 1$, quindi:
Se fosse vero dovrebbe essere: $f(n) + g(n) \leq c \min(g(n), f(n))$ ossia dovrebbe valere asintoticamente: $n + 1 \leq c$ Ossia $c \geq n + 1$. Non esiste una costante c fissata che verifichi la disuguaglianza $\forall n$.

Segue che: $f(n) + g(n) \notin \Theta(\min(g(n), f(n)))$

Esercizio 1.1.7 #OrdineGrandezza #ConfrontoFunzioni

Siano $f(n)$ e $g(n)$ due funzioni asintotiche positive. Provare o confutare la seguente congettura.

$$f(n) + g(n) = \Theta(\max(f(n), g(n)))$$

Per dimostrarlo è necessario provare che:

$$f(n) + g(n) = \begin{cases} O(\max(f(n), g(n))) \\ \Omega(\max(f(n), g(n))) \end{cases}$$

Senza perdita di generalità assumiamo che $f(n)$ sia più grande asintoticamente di $g(n)$. Ossia, $f(n) = \max(g(n), f(n))$. Cominciamo con Ω :

- $f(n) + g(n) \geq f(n) = \max(g(n), f(n))$.

Rimane da verificare O .

- $f(n) + g(n) \leq f(n) + f(n) = 2 \max(g(n), f(n))$.

Segue che: $f(n) + g(n) \in \Theta(\max(g(n), f(n)))$

Esercizio 1.1.8 Trovare un limite superiore per la ricorrenza $T(n) = \frac{n}{4} + \log_2 n$, assumendo che $T(n)$ sia costante per $n \leq 4$ e n sia una potenza di 2. Spiegare la soluzione.

1.2 Analisi di Codici Iterativi

Esercizio 1.2.1 #Iterazione

Stimare la complessità temporale della seguente porzione di codice:

```
Test( $n$  : int)
 $k := n$ ;
while  $k > 1$  do
   $k := k - 2$ ;
```

Per capire la complessità temporale della funzione TEST è necessario capire quante iterazioni il ciclo **while** svolge, e quanto è il costo computazionale delle operazioni al suo interno. Per farlo osserviamo innanzitutto che l'assegnazione al suo interno $k = k - 2$ è una operazione elementare, pertanto con costo costante $\Theta(1)$. Ora rimane da capire quante volte il suddetto ciclo viene ripetuto. Per determinarlo dobbiamo identificare:

- condizione di arresto: $k > 1$; ci fermiamo quando il valore di $k \leq 1$
- funzione crescita/decremento del contatore: $k = k - 2$;

Per individuare la funzione che si cela dietro al passo di decremento della variabile k , ci agevoliamo con la seguente tabella:

Passo	0	1	2	3	...	i
k	n	$n - 2$	$n - 4$	$n - 6$	...	$n - 2i$

Con una semplice induzione possiamo provare che passo i la funzione pone $k = n - 2i$. Per calcolare il numero di passi che il nostro ciclo dovrà svolgere chiediamoci quale è il minimo valore di i per cui $n - 2i \leq 1$.

Quindi: $i \geq \frac{n-1}{2} \implies$ la funzione TEST $\in \Theta(n)$.

Esercizio 1.2.2 #Iterazione

Calcolare il limite superiore alla complessità in tempo del seguente algoritmo.

```
Pippo( $n$  : int)
int  $t := n$ ;
int  $i := t$ ;
while  $t \geq 2$  and  $i \geq 10$  do
   $t := \lfloor \frac{t}{2} \rfloor$ ;
   $i := i - 2$ ;
```

La porzione di codice di nostro interesse è il ciclo **while**; qui abbiamo però due variabili che vengono modificate in una condizione **and**. La condizione contenuta nel predicato che invalida prima, determina il costo del ciclo. Esaminiamo quindi il numero di esecuzioni di entrambe le condizioni ^a:

Passo	1	2	3	...	k
t	n	n/2	n/4	...	n/2 ^k
$t < 2 \Rightarrow \frac{n}{2^k} < 2 \Rightarrow k \geq \log_2 n - 1$					
Quindi: $t = \log_2 n - 1 \in \Theta(\log_2 n)$					

^aPer semplicità, poiché è facile vedere che non cambia l'ordine di grandezza, omettiamo la parte intera

Passo	1	2	3	...	h
i	n	n - 2	n - 4	...	n - 2h
$i < 10 \Rightarrow n - 2h < 10 \Rightarrow h \geq \frac{n}{2} - 5$					
Quindi: $i \in \Theta(n)$					

Essendo le due condizioni in **and** nella clausola booleana, essa diventa falsa non appena una delle due condizioni è falsa. Il ciclo si interrompe dopo $\Theta(\log_2 n)$ passi e il costo dell'intero snippet di codice è $\Theta(\log n)$.

Esercizio 1.2.3 #Iterazione

Calcolare il limite superiore alla complessità in tempo dei seguenti algoritmi.

Test_1
int $t := \log_3 n$;
while $t > 1$ **do**
 $t := \frac{t}{3}$;

Test_2($A[]$: **int**, s , f :
int)
int $i := s$;
int $a := 1$;
while $i < f$ **do**
 if $A[i] \geq 0$ **then**
 $a := a * A[i]$;
 $i := i + 1$;
 else
 $i := 4i$;

Test_3($A[]$: **int**, s , f :
int)
int $i := s$;
int $a := 1$;
while $i < f$ **do**
 if $A[i] < 0$ **then**
 $a := a * A[i]$;
 $i := 2i$;
 else
 $i := f$;

Test_1:

Passo	1	2	3	...	k
t	$\log_3 n$	$\frac{\log_3 n}{3}$	$\frac{\log_3 n}{3^2}$	...	$\frac{\log_3 n}{3^k}$

Da cui abbiamo che la computazione si ferma appena:

$$t \leq 1 \Rightarrow \frac{\log_3 n}{3^k} \leq 1 \Rightarrow k = \min\{k : 3^k \leq \log_3 n\} \Rightarrow k \in O(\log \log n)$$

Test_2: Sia $n = f - s + 1$, Il caso peggiore si verifica se tutti i valori in A sono positivi. In tal caso i è incrementato di 1, e si verifica $i = f$ dopo $f - i + 1 = n$ passi. Complessità è $O(n)$.

Test_3: Supponendo che tutti i valori siano negativi, caso peggiore, abbiamo:

Passo	1	2	3	...	k
i	s	$2 \cdot s$	$2^2 \cdot s$	...	$2^k \cdot s$

Da cui abbiamo:

$$i < f \Rightarrow 2^k \cdot s < f \Rightarrow \log_2(2^k \cdot s) < \log_2(f) \Rightarrow k + \log_2 s < \log_2 f \Rightarrow$$

$$\Rightarrow k < \log_2 f - \log_2 s \Rightarrow k < \log_2 \frac{f}{s} \in O\left(\log_2 \frac{f}{s}\right)$$

Esercizio 1.2.4 #Iterazione

Studiare la complessità in tempo del seguente algoritmo iterativo:

```

Fill( Array of int A)
int  $n := \text{length}(A)$ ;
int  $z := 1$ ;
int  $i := 0$ ;
for  $i := 1; i++; n$  do
   $A[i] := 2^i$ 
for  $i := 1; i++; n$  do
   $j := A[i]$ ;
  while  $j \geq 1$  do
     $z := z + 1$ ;
     $j := \lfloor j/2 \rfloor$ ;
return:  $z$ 

```

Per stimare la complessità in tempo del codice proposto è necessario studiare la complessità dei suoi 3 cicli.

Il primo **for** parte da 1 fino ad n con passo 1.

Lo possiamo dunque esprimere come: $\sum_{i=1}^n \mathcal{O}(1) = \Theta(n)$ perché l'elevamento a potenza 2^i ha costo $\Theta(1)$.

Per quanto riguarda i restanti due cicli, il costo del secondo **for** dipende dal costo del **while** innestato all'interno del **for**.

Innanzitutto osserviamo che la condizione di arresto si raggiunge quando $j < 1$, e che il valore di j dimezza ad ogni passo. Quindi cerchiamo il minimo valore di k tale che $j/2^k < 1 = 2^0$. Otteniamo $k > \log_2 A[i] = \log_2 2^i = i + 1$ perché $A[i] = 2^i$ come da assegnazione nel primo **for**.

Passo	1	2	3	...	k
j	$j/2$	$j/4$	$j/8$	...	$j/2^k$

Precisamente, ricordando che la variabile k è una variabile intera:

$$\frac{2^i}{2^k} < 1 \quad \Rightarrow \quad 2^k > 2^i \quad \Rightarrow \quad k \log 2 > i \log 2 \quad \Rightarrow \quad k \geq i + 1$$

Da questa analisi abbiamo scoperto che il ciclo **while** più interno esegue un numero di passi $i + 1$ durante l'iterata i . Dunque il costo del nostro algoritmo sarà:

$$\Theta(n) + \sum_{i=1}^n (i + 1)\Theta(1) = \Theta(n) + \left(\frac{(n+2)(n+1)}{2} - 1 \right) \Theta(1) \Rightarrow \Theta(n^2).$$

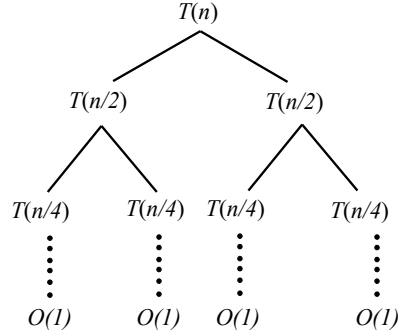
1.3 Analisi delle Equazioni di Ricorrenza

Esercizio 1.3.1 #Induzione #AlberoRicorrenze

Stimare la complessità della seguente ricorrenza e provare la stima applicando l'induzione:

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + n \log_2 n & \text{if } n > 1 \\ \Theta(1) & \text{otherwise} \end{cases}$$

Studiamone la complessità con l'albero delle ricorrenze:



Nodo generico: $T(\frac{n}{2^i})$

Livello foglie: $\frac{n}{2^i} \leq 1 \implies i \geq \log_2 n \implies i = \lceil \log_2 n \rceil$

Costo Livello generico: $2^i \cdot \frac{n}{2^i} \cdot \log_2 \frac{n}{2^i}$

Numero foglie: $2^{\log_2 n} = n$

Pertanto il costo totale può essere scritto come:

$$\sum_{i=0}^{\lceil \log_2 n \rceil - 1} \left(n \cdot \log_2 \frac{n}{2^i} \right) + n \log n + n < \sum_{i=0}^{\log_2 n} (n \cdot \log_2 n) + n \log n + n \leq 3n \log n \in O(n \cdot \log^2 n)$$

Il costo è quindi di: $O(n \cdot \log^2 n)$

Si poteva anche applicare il Master Theorem. L'equazione ricade nel secondo caso con $a = b = 2$ e $k = 1$, ossia $f(n) = n \log^k n$.

Pertanto la soluzione è $T(n) \in O(n \log^{k+1} n) = O(n \log^2 n)$.

Proviamolo per induzione:

Tesi: $T(n) \in O(n \cdot \log^2 n) \implies \exists c > 0, \exists n_0 > 0$ t.c. $\forall n \geq n_0, T(n) \leq c \cdot n \cdot \log^2 n$

- Caso Base:

Nel caso base si valuta la stima in $n = 1$. Una funzione ben definita valutata in un punto da sempre un valore finito. Pertanto $T(1)$ soddisfa la tesi.

- Passo Induttivo:

Per ipotesi induttiva e per la definizione dell'ordine di grandezza, abbiamo che:

$$T\left(\frac{n}{2}\right) \in O\left(\frac{n}{2} \cdot \log^2 \frac{n}{2}\right) \implies \exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0$$

$$T\left(\frac{n}{2}\right) \leq c \cdot \frac{n}{2} \cdot \log^2 \frac{n}{2}$$

Ora rimane di sostituire nell'equazione di ricorrenza l'ipotesi induttiva e di verificare che esiste una costante che soddisfa sia l'ipotesi induttiva che la tesi. Ossia :

$$T(n) = 2T\left(\frac{n}{2}\right) + n \cdot \log n \leq c \cdot n \cdot \log^2 \frac{n}{2} + n \cdot \log n \leq$$

$$c \cdot n \cdot (\log n - 1)^2 + n \cdot \log n = cn \cdot \log_2^2 n + \underbrace{-2cn \cdot \log n + cn + n \cdot \log n}$$

Ora per verificare la Tesi $T(n) \leq c \cdot n \cdot \log_2^2 n$ dell'induzione deve esistere una costante c che asintoticamente (ossia a partire da un certo valore n_0 in poi) rende negativa la quantità evidenziata dalla graffa. Cerchiamo una costante c tale che

$$-2cn \cdot \log n + cn + n \cdot \log n \leq 0$$

Scegliamo $c = 1$, sostituiamo e verifichiamo che per $n \geq 2$ si ha

$$-2n \cdot \log n + n + n \cdot \log n = -n \cdot \log n + n \leq 0$$

perché $\log_2 n \geq 1$ se $n \geq 2$. Pertanto, abbiamo trovato due costanti positive $c = 1$ e $n_0 = 2$ che rendono negativa la quantità indicata dalla graffa. Pertanto, la Tesi è verificata. Si noti che la costante c vale sia per l'ipotesi induttiva che per la Tesi. Si noti che il valore di c non è unico. Qualsiasi valore di c che rende negativa la quantità evidenziata dalla graffa è corretto. E' sufficiente trovarne un valore di c perché basta avere l'esistenza della costante.

Esercizio 1.3.2 #Induzione #TeoremaEsperto #IpotesiOrdineInferiore

Stimare la complessità della seguente ricorrenza e provare la stima applicando l'induzione. Si assuma che il caso base sia $T(1) = \Theta(1)$:

$$T(n) = 4T\left(\frac{n}{4}\right) + 5$$

Stimiamo innanzitutto il costo della nostra ricorrenza usando il M.T. e riconducendo l'equazione al formato $T(n) = aT(\frac{n}{b}) + f(n)$ ponendo:

$$a = 4, b = 4, f(n) = 5 \implies \#foglie = n^{\log_4 4} = n$$

Cadiamo quindi nel primo caso del M.T., ovvero la funzione $f(n)$ è più piccola del numero di foglie per un fattore polinomiale:

$$f(n) = 5 \in O(n^{\log_4 4 - \epsilon}) \text{ per una qualche } \epsilon > 0$$

Questo è verificato ad esempio scegliendo $\epsilon = 1/2$ in quanto $5 \in \Theta(1) \in O(\sqrt{n})$. Quindi $T(n) \in O(n)$.

Proviamo lo stesso risultato con induzione:

$$\exists c > 0, \exists n_0 > 0 \text{ t.c. } \forall n \geq n_0, T(n) \leq cn$$

Caso base: verificato

Ipotesi: $T(\frac{n}{4}) \leq c\frac{n}{4}$ (la prova deve essere fatta sulla forma originale), allora:

$$T(n) \leq 4c\frac{n}{4} + 5 = cn + 5$$

Da cui dovrebbe valere: $T(n) \leq cn + 5 \leq cn$

Sembrerebbe che la nostra ipotesi non sia verificata.

Possiamo però provare a ipotizzare che $T(n)$ sia una funzione lineare del tipo $T(n) = n - b$ (ossia aggiungiamo un ordine inferiore); da ciò la nuova ipotesi:

Ipotesi: $T(\frac{n}{4}) \leq c\frac{n}{4} - b$

Usiamo allora ipotesi induttiva: $T(\frac{n}{4}) \leq c\frac{n}{4} - b$.

Quindi la tesi è: $T(n) = 4cT(n/4) + 4 \leq 4(c\frac{n}{4} - b) + 5 = cn - 4b + 5$

Che facilmente verifica $cn - 4b + 5 \leq cn - b$ se e solo se esiste b costante positiva che soddisfa $5 - 3b \leq 0$.

Tale disuguaglianza è verificata per $b > 5/3$. Quindi, la soluzione $T(n) \in O(n)$ è verificata da qualsiasi costante c e $b = 2$.

Esercizio 1.3.3 #Induzione

Risolvere la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ T(\frac{n}{5}) + T(\frac{7}{10}n) + \Theta(n) & \text{if } n \geq 2 \end{cases}$$

- stimare la soluzione
- provare la soluzione per induzione

Poiché il problema è diviso in due sotto-problemi di diversa dimensione, $\frac{n}{5}$ e $\frac{7}{10} \cdot n$, non possiamo applicare il Teorema dell'Esperto; inoltre è da notare che:

$$\frac{n}{5} + \frac{7}{10}n = \frac{9}{10}n < n$$

Per cui possiamo affermare che: $T(n) \in O(n)$

Prova per Induzione:

Ipotesi Induttiva:

$$\exists c > 0, \exists n_0 > 0 : \forall n \geq n_0$$

$$T\left(\frac{n}{5}\right) \leq c \frac{n}{5}$$

$$T\left(\frac{7}{10}n\right) \leq c \frac{7}{10}n$$

Verifica della Tesi:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7}{10}n\right) + an \leq c \frac{n}{5} + c \frac{7}{10}n + an \leq \frac{9}{10}cn + an \leq cn - \frac{1}{10}cn + an \leq cn$$

L'ultima disuguaglianza è vera se: $-\frac{1}{10}cn + an \leq 0$, quindi:

$$-\frac{1}{10}cn + an \leq 0 \Rightarrow -\frac{1}{10}c + a \leq 0 \Rightarrow c \geq 10a$$

Poiché a è una costante nell'ordine $O(1)$ la prova è vera.

Esercizio 1.3.4 #TeoremaEsperto

La ricorrenza $T_F(n) = 8T_F(n/2) + n \log(n)$ descrive il tempo di esecuzione di un algoritmo F . L'algoritmo G ha invece tempo di esecuzione $T_G(n) = aT_G(n/4) + n^2$.

- Risolvere l'equazione T_F
- Risolvere l'equazione T_G al variare del parametro a
- Confrontare le complessità dei due algoritmi al variare del parametro a .

Per calcolare la complessità della ricorrenza T_F possiamo avvalerci del Master Theorem. Infatti, nell'equazione data abbiamo $a = 8, b = 2, f(n) = n \log n$. Possiamo facilmente osservare che la nostra ricorrenza cade nel primo caso del Master Theorem, in quanto $\log_b a = \log_2 8 = 3$; da questo è facile dimostrare la dominanza asintotica del costo $f(n)$ delle foglie sul costo del passo "combina". Prendendo ad esempio $\epsilon = 1$ è facile dimostrare che

$f(n) = n \log n \in O(n^3 - \epsilon)$. Da questo segue che $T_F(n) \in \Theta(n^{\log_b a} = n^3)$.

Per quanto riguarda il punto b) dell'esercizio, dobbiamo studiare al variare del parametro a come cambia la complessità della ricorrenza $T_G(n) = aT_G(n/4) + n^2$.

Anche in questo caso possiamo sfruttare il Master Theorem per dedurre facilmente i costi asintotici che la ricorrenza T_G assumerà al variare di a .

In particolare abbiamo che $b = 4, f(n) = n^2$, quindi possiamo innanzitutto osservare che quando $n^{\log_4 a} = n^2$, quindi quando $a = 16$, cadiamo nella seconda casistica del Master Theorem, con una complessità asintotica di $\Theta(n^2 \log n)$.

Mentre quando $1 \leq a < 16$, possiamo ricondurci al terzo caso del Master Theorem con una complessità asintotica di $O(f(n)) = O(n^2)$. Segue che per valori di $a > 16$ si cade nel primo caso del Master Theorem con una complessità asintotica di $O(n^{\log_4 a})$.

Da quanto appena osservato, abbiamo che la funzione T_F è computazionalmente più pesante della funzione T_G per tutti i valori di a tali per cui $\log_3 a < 3$, quindi $a < 64$. Ossia $T_F \in \Omega(T_G)$.

Quindi: Si equivalgono asintoticamente quando $a = 64$, mentre per tutti i valori $a > 64$ la funzione T_G domina la funzione T_F , ossia $T_G \in \Omega(T_F)$. Similmente se $a < 64$, la funzione T_G è dominata da T_F , ossia $T_G \in O(T_F)$.

1.4 Analisi di Codici Ricorsivi

Esercizio 1.4.1 #Ricorsione #AlberoRicorrenze

Calcolare il limite superiore alla complessità in tempo del seguente algoritmo. E' sufficiente stimare la soluzione dell'equazione di ricorrenza, senza dare la prova per induzione.

```

Fun ( $n$  : int)
if  $n < 4$  then
  | return: 1;
else
  | if  $n < 2^{10}$  then
    |  $i := 0$ ;
    | while  $i < n$  do
      |  $i := i + 2$ ;  $a := a * 5$ ;
    | return:  $a$ ;
  | else
    | int  $t := 2$ ;
    | while  $t \leq n$  do
      |  $t := t * 2$ ;
    | Fun := Fun( $\frac{n}{4}$ ) + Fun( $\frac{n}{4}$ ) +  $\sqrt{n}$ ;

```

Il codice è ricorsivo. Vi sono due rami che sono limitati da un valore finito di n . Tali rami, se terminano, hanno sempre complessità $O(1)$, infatti qualsiasi funzione di n con n limitato superiormente appartiene a $\Theta(1)$. Il costo asintotico è dato dal ramo **else** del secondo **if** dove la variabile t inizializzata a 2 viene raddoppiata finché minore di n , con un costo totale che vedremo essere di $O(\log(n))$ operazioni.

Calcoliamo quante volte il corpo del **while** è eseguito con lo schema seguente:

Passo	1	2	3	4	...	s
t	2	4	8	16	...	2^s

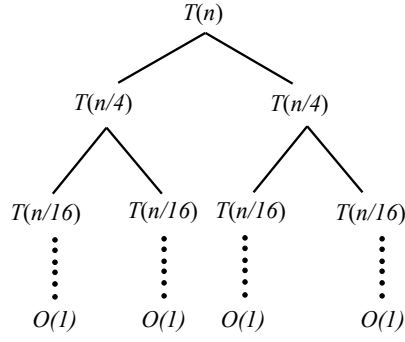
Da cui il ciclo è eseguito finché:

$$t \leq n \rightarrow 2^s \leq n \rightarrow s \cdot \log_2 2 \leq \log_2 n \rightarrow s \leq \log_2 n \rightarrow O(\log n)$$

Dunque l'equazione di ricorrenza che corrisponde è la seguente:

$$T(n) = \begin{cases} 2T(\frac{n}{4}) + \Theta(\log n), & n \geq 2^{10} \\ \Theta(1), & n < 2^{10} \end{cases}$$

Stimiamo il costo con l'albero di ricorrenza:



Nodo generico: $T(\frac{n}{4^i})$

Altezza: $\frac{n}{4^i} < 2^{10} \implies i > \log_4 n - 5\Theta(\log_4 n)$

Costo Livello generico: $2^i \cdot \log_2 \frac{n}{4^i}$

Numero foglie: $2^{\log_4 n} = n^{\frac{1}{2}}$

Osservato che $\log_4 n - 5\Theta(\log_4 n) \in O(\log_4 n)$, il costo totale è limitato superiormente da:

$$\begin{aligned} \sum_{i=1}^{\log_4 n - 1} \left(2^i \cdot \log \frac{n}{4^i} \right) + n^{\frac{1}{2}} &< \sum_{i=1}^{\log_4 n - 1} (2^i \cdot \log n) + \sqrt{n} = \\ &= 2^{\log_4 n} \cdot \log n + \sqrt{n} = \sqrt{n} \cdot \log n + \sqrt{n} \in O(\sqrt{n} \cdot \log n) \end{aligned}$$

Il costo è : $O(\sqrt{n} \cdot \log n)$. Attenzione, il costo è limitato superiormente, quindi O , in quanto abbiamo maggiorato il costo di ciascun livello con un limite non stretto.

Avremmo potuto risolvere questo problema anche usando il Teorema dell'Esperto, siamo infatti nel primo caso:

$$f(n) = \log n \in O(n^{\log_4(2) - \epsilon}) \text{ con } \epsilon > 0$$

Con $0 < \epsilon < \frac{1}{2}$ abbiamo che è vero, quindi il costo è: $\Theta(n^{\log_4(2)}) = \Theta(\sqrt{n})$

Quindi usando il Teorema dell'Esperto troviamo un limite superiore più stretto.

Esercizio 1.4.2 #Ricorsione #AlberoRicorrenze

Calcolare il limite superiore alla complessità in tempo dei seguenti algoritmi. E' sufficiente stimare la soluzione dell'equazione di ricorrenza, senza dare la prova per induzione.

```

Pluto ( $n : \text{int}$ )
if  $n \geq 10$  then
  | Pluto := Pluto( $\frac{n}{3}$ ) + Pluto( $\frac{n}{3}$ )
  |   +  $2\sqrt{n}$ ;
else
  | if  $n \geq 2$  then
  |   | Pluto := 7;
  |   else
  |     | Pluto := Pluto( $\frac{n}{2}$ ) + 100;

```

```

Pippo( $n : \text{int}$ )
if  $n \leq 3$  then
  | return 3;
else
  | int  $a := n$ ; int  $j := 0$ ;
  | while  $a > 2$  do
  |   | while  $j < 5$  do
  |     |  $j := j + 1$ ;
  |     |  $a := \frac{a}{8} * j$ ;
  |   Pippo :=  $4 \cdot \text{Pippo}(\frac{n}{2}) +$ 
  |     Pippo( $\frac{n}{2}$ );

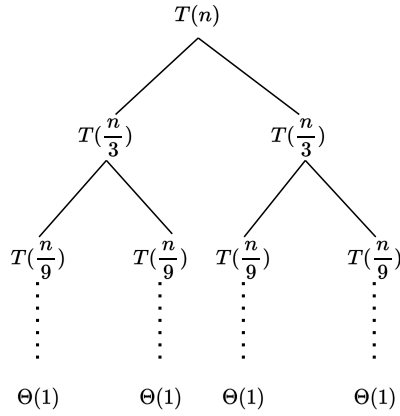
```

Analizziamo la porzione di codice **(a)**: Notiamo che il ramo asintotico della funzione ricorsiva **PLUTO** è il primo ramo che si esegue se $n \geq 2$.

Dunque l'equazione di ricorrenza che corrisponde è la seguente:

$$T(n) = \begin{cases} 2T(\frac{n}{3}) + \Theta(1), & n \geq 2 \\ \Theta(1), & n < 2 \end{cases}$$

Risolviamo con albero della ricorsione come segue:



Nodo generico: $T(\frac{n}{3^i})$

Altezza: $\frac{n}{3^i} \leq 2 \implies i \geq \log_3 \frac{n}{2}$

Costo livello generico: $2^i \Theta(1)$

Numero foglie: $2^{\log_3 \frac{n}{2}} \in O(n^{\log_3 2})$

Pertanto il costo totale può essere scritto come:

$$\sum_{i=1}^{\log_3 \frac{n}{2} - 1} 2^i \Theta(1) + n^{\log_3 2} \in O(n^{\log_3 2})$$

Il ramo asintotico del codice ricorsivo **(b)** è il ramo **else**. Innanzitutto valutiamo il costo dei due cicli **while** annidati.

- Il **while** interno esegue sempre 5 passi $\implies \Theta(1)$
- Il **while** esterno si ferma quando la variabile $a \leq 2$ e la stessa viene decrementata con passo $\frac{5}{8}$; questo perché si ha $j = 5$ ogni volta che si decrementa la dimensione di a .

Studiamo come varia la dimensione di a :

Passo	0	1	2	3	...	k
a	n	5n/8	25n/64	125n/512	...	$n(5/8)^i$

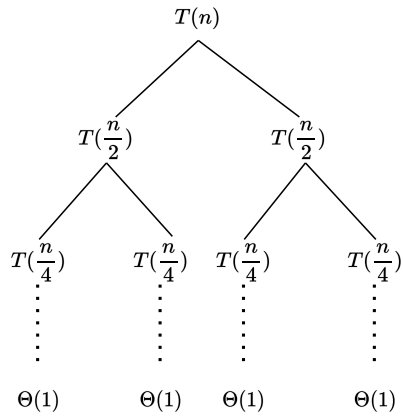
Quindi:

$$n \left(\frac{5}{8}\right)^i \leq 2 \implies i \geq \log_{\frac{5}{8}} \frac{n}{2} \in \Theta(\log n)$$

Pertanto la ricorrenza può essere scritta come:

$$T(n) = \begin{cases} 2T(\frac{n}{2}) + \Theta(\log n), & n \geq 3 \\ \Theta(1), & n < 3 \end{cases}$$

Ancora una volta usiamo l'albero di ricorsione:



Nodo generico: $T(\frac{n}{2^i})$

Altezza: $\frac{n}{2^i} \leq 3 \implies i \geq \log_2 \frac{n}{3}$

Costo livello generico: $2^i \log_{\frac{8}{5}} \frac{n}{2^i}$

Numero foglie: $2^{\log_2 \frac{n}{3}} = (\frac{n}{3})^{\log_2 2} = \frac{n}{3}$

Pertanto il costo totale può essere scritto come:

$$\sum_{i=1}^{\log_2 \frac{n}{3} - 1} 2^i \log_{\frac{8}{5}} \frac{n}{2^i} + \frac{n}{3} \leq \sum_{i=1}^{\log_2 \frac{n}{3} - 1} 2^i \log_{\frac{8}{5}} n + \frac{n}{3} = \log_{\frac{8}{5}} n \left(\frac{2^{\log_2 \frac{n}{3} - 1} - 1}{2 - 1} \right) + \frac{n}{3} = \frac{n}{3} \log_{\frac{8}{5}} n - \log_{\frac{8}{5}} n + \frac{n}{3}$$

Quindi nell'ordine di grandezza: $O(n \log n)$

Esercizio 1.4.3 #AlberoRicorrenze #Iterazione #Ricorsione

Risolvere con l'albero della ricorsione l'equazione di ricorrenza che rappresenta la complessità in tempo del seguente algoritmo. Provare per induzione la stima fatta.

```

Pippo( int n)
if  $n \leq 5$  then
  return 3;
else
  int  $i := 1$ ;
  while  $i \leq n$  do
    for  $j := 1; j++ ; j \leq n$  do
       $i := i + 3$ ;
  Pippo = Pippo( $\frac{n}{5}$ ) + Pippo( $\frac{n}{7}$ );

```

Per stimare la complessità in tempo del codice proposto è necessario identificare l'equazione di ricorrenza. Quindi per prima cosa studiamo il costo del passo combina, quindi i due cicli annidati.

Consideriamo il costo del **for** più interno.

Esso parte da 1 fino ad n con passo 1, ha quindi costo: $\sum_{j=1}^n O(1) = \Theta(n)$

Studiamo ora come cresce la variabile i .

Essa viene incrementata di 3 ad ogni iterazione del ciclo **for**. Pertanto, al termine del **for** più interno (n passi) la variabile i avrà raggiunto il valore $3n + 1$ che è un valore maggiore di n ; ciò implica che il ciclo **while** uscirà dopo una sola iterazione.

Possiamo quindi definire l'equazione di ricorrenza:

$$T(n) = \begin{cases} T(n/5) + T(n/7) + \Theta(n) & \text{if } n > 5 \\ \Theta(1) & \text{if } n \leq 5 \end{cases}$$

Dall'equazione appena scritta è facile osservare che si avrà un albero binario sbilanciato. Dunque procediamo con la maggiorazione $T'(n)$ per semplificare la stima di un upper-bound alla nostra ricorrenza.

$$T'(n) = \begin{cases} 2T'(n/5) + O(n) & \text{if } n > 5 \\ \Theta(1) & \text{if } n \leq 5 \end{cases}$$

Costruiamo l'albero di ricorrenza:

Nodo generico: $T(n/5^i)$

Altezza: $\frac{n}{5^i} \leq 5 \Rightarrow i \geq \log_5 n - 1$

Costo livello generico: $2^i (\frac{n}{5^i})$

Numero foglie: $2^{\log_5 n} = (n)^{\log_5 2}$

Possiamo quindi stimare un costo per la nostra ricorrenza:

$$\sum_{i=0}^{\log_5 n - 1} (2^i \frac{n}{5^i}) + n^{\log_5 2} < n \sum_{i=1}^{\infty} (\frac{2}{5})^i + n^{\log_5 2} = \frac{5}{3}n + n^{\log_5 2} \Rightarrow \in O(n)$$

Proviamo il risultato ottenuto per induzione sull'equazione di ricorrenza originale:

Ipotesi: $\exists c > 0, n_0 > 0 : \forall n \geq n_0 \quad T(n) \in O(n)$

Abbiamo due ipotesi induttive:

$$T\left(\frac{n}{5}\right) \leq c\frac{n}{5} \text{ e } T\left(\frac{n}{7}\right) \leq c\frac{n}{7}$$

$$T(n) \leq c\frac{n}{5} + c\frac{n}{7} + O(n) \quad \Rightarrow \quad T(n) \leq c\frac{12n}{35} + c_1n$$

In particolare:

$$c\frac{12n}{35} + c_1n \leq cn \quad \Rightarrow \quad cn\left(1 - \frac{12}{35}\right) \geq c_1n \quad \Rightarrow \quad cn\left(\frac{23}{35}\right) \geq c_1n \quad \Rightarrow \quad c \geq \frac{35}{23}c_1 \quad T(n) \in O(n)$$

Capitolo 2

Algoritmi di Ordinamento

2.1 Ordinamento

Esercizio 2.1.1 #Ottimalità

Dati tre array A , B , e C , rispettivamente ordinati in ordine crescente e la cui ultima chiave è $+\infty$, si progetti un algoritmo in grado di creare un quarto array D composto dagli elementi di A , B , e C , garantendo che sia anch'esso ordinato in modo crescente.

- Descrivere l'algoritmo a parole;
- Dare lo pseudocodice dell'algoritmo;
- Studiare la complessità dell'algoritmo proposto e discutere l'ottimalità.

Elaborazione Soluzione: Per semplicità assumiamo che $A.LENGTH() = n$, $B.LENGTH() = m$, e

$C.LENGTH() = k \implies D.LENGTH() = n + m + k - 3 + 1$. Infatti i tre array sono completati da un'ultima posizione che contiene la chiave sentinella $+\infty$, e lo stesso deve accadere per D . La sentinella ci permette di controllare se un qualche array è finito.

Per risolvere tale problema potevamo applicare un approccio *brute-force* che consiste nell'unire i 3 all'interno di D e poi richiamare una procedura di ordinamento, MERGE-SORT, o HEAP-SORT, con un costo asintotico di $O(n \log n)$.

Ovviamente questa strategia non è ottimale, visto che non sfrutta l'informazione data in input, ovvero che i tre array sono al loro interno ordinati ed è ovviamente più facile creare un array ordinato a partire da tre array ordinati che da tre array le cui chiavi appaiono in un ordine qualsiasi.

Una possibile strategia per risolvere in modo efficiente il nostro problema potrebbe essere quella di scrivere una procedura MERGE simile a quella del MERGE-SORT, ma idonea per gestire contemporaneamente 3 array.

Pseudocodice:

Merge-3wise(Array A, Array B, Array C)

Array $D[A.LENGTH() + B.LENGTH() + C.LENGTH() - 3 + 1] = \{0 \dots 0\}$

int $i = 0, j = 0, z = 0, curr = 0$

while $curr \leq D.LENGTH()$ **do**

if $A[i] \leq B[j]$ **and** $A[i] \leq C[z]$ **then**

$D[curr] = A[i]$

$i = i + 1$

else

if $B[j] \leq A[i]$ **and** $B[j] \leq C[z]$ **then**

$D[curr] = B[j]$

$j = j + 1$

else

$D[curr] = C[z]$

$z = z + 1$

$curr = curr + 1$

return: D

Complessità: L'algoritmo proposto non fa altro che tenere traccia dell'elemento corrente più piccolo per ogni array, e stabilendone la relazione con i rimanenti 2 elementi si va ad inserire nella prima posizione disponibile nell'array D risultato.

Visto che ad ogni passo inseriamo un elemento, la complessità dell'algoritmo sarà data dalla somma delle dimensioni dei 3 array, i.e., $n + m + k$, ora se assumiamo che $n \geq m \geq k$, possiamo concludere che al più paghiamo $3n$ passi $\implies \in \Theta(n)$.

Ottimalità: l'algoritmo è ottimo perché per inserire n elementi in n posizioni è necessario scandire tutti gli n elementi almeno una volta, e l'algoritmo MERGE-3WISE fa esattamente questo, quindi è ottimo.

Esercizio 2.1.2 #Ottimalità #RicercaBinaria

Un array ordinato in senso crescente $A[0 : n - 1]$ di dimensione n , i cui elementi sono tutti distinti, si dice shiftato di $k > 0$ posizioni verso destra se dopo lo shift l'array shiftato A' soddisfa la seguente condizione: $A'(i + k) \bmod n = A[i]$ per ogni $i = 0, \dots, n - 1$.

- Dato A' , scrivere un algoritmo ottimo per determinare k .
- Progettare un algoritmo in pseudo-codice, discutere la complessità in tempo e l'ottimalità.

Esempio:

Input: $A = \{2, 5, 7, 8, 10\}$, $A' = \{8, 10, 2, 5, 7\}$

Output: $k = 2$

Elaborazione Soluzione: Siano i ed f l'indice iniziale e finale dell'array A' ; inizialmente $i = 0$ e $f = n - 1$. Prima dello shift abbiamo che $A[i] < A[f]$ e dopo lo shift $A[i] > A[f]$. Questa condizione ci aiuta a trovare la soluzione.

Pseudocodice:

Consideriamo la dimensione degli array A ed A' , n , come un parametro globale

Find-Shift(Array A' , int i , int f)

// Finché non restiamo con 3 valori continuiamo a dimezzare lo spazio di ricerca

if $(f - i + 1) > 3$ **then**

$q = \left\lfloor \frac{i+f}{2} \right\rfloor$

if $A'[i] < A'[q]$ **then**

Find-Shift(A' , q , f)

else

Find-Shift(A' , i , q)

else

 // Cerchiamo il punto tra i 3 rimasti

$t = i$

while $(t < f)$ **and** $(A'[t] < A'[t + 1])$ **do**

$t = (t + 1)$

return: $(t + 1)$

La prima chiamata è: **FIND-SHIFT**(A' , 0, n).

Si noti che lo shift deve essere un numero di posizioni strettamente minore di n , per cui nel caso base non serve l'operazione modulo n . Inoltre, il codice gestisce

Complessità: Sfruttando il fatto che A' è ordinato, cerchiamo k guardando o solo sul lato destro o quello sinistro della sezione dell'array in esame; quindi impieghiamo un tempo $\Theta(\log n)$, dove n è la dimensione dell'array.

Questo tempo è ottimo perché lo spazio delle soluzioni consiste dei valori $1 \leq k \leq n - 1$. Pertanto, la tecnica dell'albero delle decisioni ci dice che per discernere tra tali soluzioni dobbiamo fare almeno $\Omega(\log n)$ scelte.

Esercizio 2.1.3 #Ottimalità

Dato un array A di cardinalità n contenente numeri interi, costruire un array B di cardinalità n così definito: $B[i]$ è la lunghezza del sotto-array (array costituito da elementi consecutivi) non decrescente più lungo in $A[1], \dots, A[i]$, per $1 \leq i \leq n$.

- Progettare e dare lo pseudocodice dell'algoritmo proposto.
- Analizzare la complessità in tempo della soluzione proposta e discuterne l'ottimalità.

Elaborazione Soluzione:

Esempio: $A = [4, 5, 5, 5, 6, 0, 1]$, allora $B = [1, 2, 3, 4, 5, 5, 5]$.

Pseudocodice:

Longest-Subsequence(Array $A[n]$)

Array $B[n] = \{0 \dots 0\}$

int $counter = 0$ // Contatore lunghezza della sotto-sequenza

int $max = 1$ // Lunghezza della più lunga sotto-sequenza trovata

for $i = 1, i \leq n, i++$ **do**

if $i == 1$ **then**

$counter++$

$B[i] = counter$

else

if $A[i-1] \leq A[i]$ **then**

$counter++$

if $counter > max$ **then**

$B[i] = counter$

$max = counter$

else

$B[i] = B[i-1]$

else

$counter = 1$

$B[i] = B[i-1]$

return: B

Complessità: La soluzione ha costo $O(n)$ ed è ottima perchè dobbiamo generare B di dimensione n .

Esercizio 2.1.4 #CountingSort

Sia dato un insieme A di n interi, ciascuno compreso fra 1 e $2n$ (ossia, per ogni $1 \leq i \leq n$, $1 \leq A[i] \leq 2n$).

Si discuta quale algoritmo fra gli algoritmi visti a lezione è il più efficiente in tempo per ordinare A . Dare lo pseudocodice di tale algoritmo.

Si risponda inoltre alle seguenti domande:

- L'algoritmo proposto è "*in-place*" (sul posto), quindi con memoria costante?
- E' esso *stabile*?

Giustificare le risposte.

Sapendo che gli elementi di A variano in un range di valori, possiamo applicare l'algoritmo COUNTINGSORT; il cui costo è:

$$O(\text{dimensione vettore} + \text{range dei valori}) = (n + 2n) \in O(n).$$

Pseudocodice CountingSort:

CountingSort(Array $V[n]$, int k)

Array $U[n] = \{0 \dots 0\}$

Array $K[2n] = \{0 \dots 0\}$

for $i = 1; i \leq n; i++$ **do**

$K[V[i]]++$
 // $K[i]$ contiene il numero di elementi uguali ad i

for $i = 1; i \leq k; i++$ **do**

$K[i] = K[i] + K[i-1]$ // $K[i]$ contiene il numero di elementi minori od uguali ad i

for $i = n; i \geq 1; i--$ **do**

 // copiamo V dentro ad U iniziando dal fondo
 $U[K[V[i]]] = V[i]$
 $K[V[i]]--$ // diminuiamo $K[V[i]]$ perchè rimane un valore in meno minore o
 uguale a $V[i]$ da sistemare in U

return: U

Osservazione: Se il range fosse stato fra 1 e n^2 è ancora possibile avere un algoritmo lineare, ma non utilizzando *Counting Sort*. L'algoritmo ottimo in questo caso è RADIXSORT. Infatti se ogni chiave si interpreta in base n , ogni chiave ha al più 2 cifre, ciascuna nel range $0, \dots, n-1$. Quindi per ordinare l'array è sufficiente invocare 2 volte counting sort, una volta per ciascuna cifra.

Risposte alle Domande:

- COUNTINGSORT non è "in-place" perché utilizza memoria aggiuntiva $O(n)$.
- Dipende dall'implementazione: l'implementazione classica qui descritta è stabile in quanto occorrenze della stessa chiave hanno nell'array in input e in quello in output U la stessa posizione relativa. Inoltre il fatto di assegnare a ciascuna entry di U il valore contenuto in $V[i]$ permette di mantenere la stabilità dell'algoritmo anche quando alla chiave sono associati anche dati satelliti.

La soluzione di counting sort che segue non è stabile. Semplicemente replica ogni chiave tante volte quante essa appare nell'array originale. L'array in output è ordinato.

CountingSortSemplificato(Array $V[n]$, int k)

Array $U[n] = \{0 \dots 0\}$

Array $K[2n] = \{0 \dots 0\}$

for $i = 1; i \leq n; i++$ **do**

$K[V[i]]++$
 // $K[i]$ contiene il numero di elementi uguali ad i //

$t := 1;$

for $i = 1; i \leq 2n; i++$ **do**

for $j = 1; j \leq K[i]; j++$ **do**
 $U[t] = i$
 $t++$

return: U

Esercizio 2.1.5 #Ottimalità #CountingSort

Dato un array P disordinato di palline colorate, i cui colori possono variare nell'insieme $\text{COLOR} = \{R, G, B\}$ ($R = \text{RED}, G = \text{GREEN}, B = \text{BLUE}$), dare un algoritmo che ordini l'array P posizionando

prima le palline RED, poi le GREEN, e infine le BLUE. Dare lo pseudocodice e discutere l'ottimalità della soluzione proposta.

Analisi del Problema: Il problema richiede di ordinare elementi secondo un ordine specifico predefinito: RED < GREEN < BLUE. Questo è un caso particolare di ordinamento con un dominio limitato (solo 3 valori possibili).

Elaborazione Soluzione: Possiamo risolvere il problema in due modi:

1. *Approccio con dizionario:* convertire ogni colore in un numero (RED = 1, GREEN = 2, BLUE = 3), ordinare il vettore risultante e fare la conversione inversa per ottenere di nuovo un array di colori.
2. *Approccio con conteggio:* Contare il numero di RED (n_R), GREEN (n_G) e BLUE (n_B). Dopodiché creare un nuovo array dove mettiamo prima n_R elementi di RED, poi n_G elementi di GREEN e infine n_B elementi di BLUE.

Nota importante: La prima soluzione che proponiamo è un algoritmo **stabile** che sfrutta la versione stabile di COUNTING-SORT. Successivamente si proporrà una seconda che è utilizzabile se non ci sono dati satelliti associati ai nostri colori.

A scopo didattico riporteremo qui entrambe le soluzioni.

• **Prima Soluzione** (*Approccio con dizionario*):

```

numbers2colors(Array  $V[n]$ )
for  $i = 1; i \leq n; i++$  do
    if  $V[i] == 1$  then
         $V[i] = \text{RED}$ 
    else
        if  $V[i] == 2$  then
             $V[i] = \text{GREEN}$ 
        else
             $V[i] = \text{BLUE}$ 
return:  $V$ 

```

```

colors2numbers(Array  $V[n]$ )
for  $i = 1; i \leq n; i++$  do
    if  $V[i] == \text{RED}$  then
         $V[i] = 1$ 
    else
        if  $V[i] == \text{GREEN}$  then
             $V[i] = 2$ 
        else
             $V[i] = 3$ 
return:  $V$ 

```

```

RGB-sorting(Array  $V[n]$ )
 $V = \text{COLORS2NUMBERS}(V)$ 
COUNTING-SORT( $V$ , RANGE(1,3))
 $V = \text{NUMBERS2COLORS}(V)$ 
return:  $V$ 

```

Analisi della Complessità: La complessità di COLORS2NUMBERS e NUMBERS2COLORS è $\Theta(n)$, dato che in entrambe si scorre al più tutto l'array una sola volta. La funzione RGB-SORTING ordina i colori sfruttando l'algoritmo di ordinamento COUNTING-SORT. Questo algoritmo ha un costo lineare ($\Theta(n + k)$), dove k è la dimensione del range di valori ed n quella del vettore in input. Poiché la dimensione del nostro range è 3, il costo del nostro algoritmo è quindi $O(n)$.

Ottimalità: l'algoritmo RGB-SORTING è **ottimo** perché per ordinare l'array dato in input dobbiamo valutare ciascun elemento almeno una volta. Per questo motivo, qualunque algoritmo richiede almeno $\Omega(n)$ passi. Segue per questo motivo l'ottimalità.

Osservazione: Si tenga a mente che in questo esercizio l'algoritmo di ordinamento COUNTING-SORT non deve subire alcuna modifica rispetto all'originale. Pertanto, lo invocheremo senza riportare la sua definizione. Si rimanda il lettore alla Sezione 8.2 del libro [1] per spiegazione completa di COUNTING-SORT.

• **Seconda Soluzione** (*Approccio con conteggio*):

<pre> countingColors(Array $V[n]$) int $n_R = 0, n_G = 0, n_B = 0$ for $i = 1; i \leq n; i++$ do if $V[i] == \text{RED}$ then n_R++ else if $V[i] == \text{GREEN}$ then n_G++ else n_B++ return: n_R, n_G, n_B </pre>	<pre> makeSolution(Array $V[n]$) int $n_R, n_G, n_B =$ countingColors(V) Array $U[n] = \emptyset$ for $i = 1; i \leq n_R; i++$ do $U[i] = \text{RED}$ for $i = n_R + 1; i \leq n_G; i++$ do $U[i] = \text{GREEN}$ for $i = n_G + 1; i \leq n_B; i++$ do $U[i] = \text{BLUE}$ return: U </pre>
---	--

Analisi della Complessità: La soluzione proposta sfrutta due funzioni COUNTINGCOLORS e MAKESOLUTION, rispettivamente. Pertanto dobbiamo studiare la loro complessità per dedurre il costo dell'intera soluzione.

Cominciamo con COUNTINGCOLORS. Questa funzione conta quante volte i colori RED, GREEN e BLUE compaiono all'interno dell'array scorrendo ogni elemento una e una sola volta. Segue quindi che il costo sia $n \in O(n)$.

Si continua valutando MAKESOLUTION. Questa funzione invoca COUNTINGCOLORS ottenendo le occorrenze per ognuno dei tre colori (n_R, n_G, n_B). Dopodiché procede con la creazione dell'array risultato andando ad assegnare i colori alle rispettive celle sulla base del numero di occorrenze e dell'ordine richiesto dall'esercizio. Riassumendo, abbiamo un costo complessivo di $O(n + \underbrace{n_R + n_G + n_B}_{=n}) \Rightarrow O(n)$.

Dato che il costo asintotico dell'algoritmo è $O(n)$, segue la sua ottimalità per le stesse considerazioni riportate sopra per la precedente soluzione.

Esercizio 2.1.6 #MergeSort

Sia dato un insieme A di n numeri reali, ciascun reale è compreso fra 1 e n (ossia, per ogni $1 \leq i \leq n, A[i] \in \mathbb{R}, 1 \leq A[i] \leq n$). Si discuta quale algoritmo fra gli algoritmi visti a lezione è il più efficiente in tempo per ordinare A .

Il problema presenta le seguenti caratteristiche:

- **Tipo di dati:** Numeri reali ($\mathbb{R}$)
- **Range:** Valori compresi nell'intervallo $[1, n]$
- **Cardinalità del dominio:** Infinita (continuum dei reali)
- **Dimensione input:** n elementi

Per effettuare la scelta dell'algoritmo più adatto, dobbiamo ricordare che gli algoritmi di ordinamento si dividono in due categorie principali:

- **Algoritmi Comparison-Based**
- **Algoritmi Non Comparison-Based**

A riguardo dei primi è necessario ricordare che:

Principio: Ordinano confrontando coppie di elementi.

Limite Inferiore: $\Omega(n \log n)$ nel caso pessimo.

Esempi:

- MERGESORT: $\Theta(n \log n)$ tempo
- HEAPSORT: $\Theta(n \log n)$ tempo
- QUICKSORT: $\Theta(n^2)$ tempo,

Per quanto riguarda la categoria degli **algoritmi Non Comparison-Based**:

Principio: Sfruttano proprietà specifiche del dominio dei dati.

Potenziati vantaggi: Possono raggiungere complessità lineare $O(n)$ sotto certe condizioni.

Esempi:

- COUNTINGSORT: $\Theta(n + k)$ dove k è la dimensione del range, ossia il numero di valori discreti che possono essere assunti dai valori da ordinare
- RADIXSORT: $\Theta(d \cdot (n + k))$ dove d è il numero di cifre in base k del numero più grande nell'insieme da ordinare

Analisi delle Opzioni

Si comincia osservando che COUNTINGSORT non è direttamente applicabile. **Requisito fondamentale:** COUNTINGSORT richiede che il dominio dei valori sia *discreto* e di cardinalità *finita*.

Problema nel nostro caso:

- Il dominio è $[1, n] \cap \mathbb{R}$ che ha cardinalità $|\mathbb{R}| = \aleph_1$ (continuum)
- COUNTINGSORT richiederebbe un array di conteggio di dimensione infinita perchè i numeri da ordinare sono reali

Si continua osservando che nemmeno RADIXSORT è direttamente applicabile in quanto applica counting sort.

Algoritmi che ordinano in modo ottimo sono **MergeSort** e **HeapSort** la cui complessità è $\Theta(n \log n)$.

Capitolo 3

Divide ed Impera

3.1 Ricerca di Elementi nelle Matrici

Esercizio 3.1.1 #RicercaBinaria, #Ottimalità

Data una matrice M di dimensione $m \times m$ i cui elementi sono uguali a 0 o 1. Siano i suoi elementi ordinati in modo che visitando la matrice per colonne con indice da 1 a m e ogni colonna dall'alto verso il basso, si incontrino prima tutti gli 0's e poi gli 1's. Vi sono almeno uno 0 ed in 1 nella matrice M . Dare un algoritmo per trovare la posizione del primo 1 in M .

- Dare lo pseudocodice dell'algoritmo;
- Studiare la complessità dell'algoritmo proposto e discutere l'ottimalità.

Esempio:

input: $M = 4 \times 4$

$$M = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

output: (2,3)

Elaborazione Soluzione n.1: L'esercizio ci richiede di trovare il primo 1 nella nostra matrice M assumendo di scorrerla per colonne. La soluzione più semplice che si poteva descrivere per risolvere il nostro problema consiste nello scorrere partendo dal primo elemento della prima colonna di M tutte le successive colonne. Questo significa fare una visita esaustiva dell'intera matrice; quindi una soluzione a forza bruta. Nel testo dell'esercizio, si assume che il numero di righe sia uguale al numero di colonne e sia denominato m . La soluzione qui proposta vale che per matrici rettangolari.

Pseudocodice:

```
Brute-Force(Matrix  $M[m \times m]$ )
int  $i = 1$ 
int  $j = 1$ 
while  $i < m$  do
    while  $j < m$  do
        if  $M[j][i] == 1$  then
            return: ( $i, j$ )
         $j++$ 
     $i++$ 
return: -1
```

Complessità: La funzione BRUTE-FORCE fa una ricerca esaustiva all'interno della matrice muovendosi per colonne. Il caso peggiore per il nostro algoritmo è quando non vi è la presenza di alcun 1, e in quel caso scorriamo tutta la matrice con un costo temporale di $O(m^2)$.

Elaborazione Soluzione n.2: Assumiamo che di avere a disposizione una funzione per la ricerca binaria $BINARY-SEARCH(A, 1, m)$ che restituisce l'indice del primo 1, se presente, nell'array A e -1 altrimenti.

```

Smart-Scan(Matrix  $M[m \times m]$ )
int  $i = 1$ 
int  $j = 1$ 
 $j = \text{BINARY-SEARCH}(\text{row}[1], 1, m)$  //  $(1, j)$  è la posizione del
    primo 1 nella prima riga di  $M$ 
 $i = \text{BINARY-SEARCH}(\text{col}[j - 1], 1, m)$ 
if  $i = -1$  then
    | return:  $(1, j)$ 
    | // No 1 in column  $j - 1$ 
else
    | return:  $(i, j - 1)$ 

```

Elaborazione Soluzione n.2: La prima ricerca binaria sulla prima riga di M ritorna la prima colonna che inizia con 1. Tale colonna esiste ed ha indice $j \geq 2$. Il primo uno si trova quindi nella colonna $j - 1$ se la ricerca

Osserviamo che un piccolo trucco ci permette di gestire facilmente anche il caso in cui non sia garantito che M abbia almeno uno 0 ed un 1. Se invochiamo la prima ricerca binaria sull'ultima riga se ritorna -1 vuol dire che non esiste alcun 1 in M . Una volta trovata la colonna dobbiamo necessariamente valutare dove si trova il primo 1 in tale colonna.

La funzione SMART-SCAN ha costo computazionale pari a $O(\log m)$.

Ottimalità: l'algoritmo SMART-SCAN è ottimo perché lo spazio delle soluzioni, ossia le posizioni candidate a contenere il primo 1 ha cardinalità m^2 , e la tecnica dell'albero delle decisioni ci dice che per discernere fra m^2 soluzioni abbiamo bisogno di almeno $\Omega(\log_2 m^2) = \Omega(\log m)$.

Quindi SMART-SCAN avendo costo $O(\log m)$ è un algoritmo ottimo. In sintesi, scriviamo la complessità di SMART-SCAN è $\Theta(\log m)$.

3.2 Ricerca di Elementi nei Vettori

Esercizio 3.2.1 #RicercaBinaria #Ottimalità

Dato un array A ordinato di dimensione n contenente esclusivamente 2 e 3, si definisca un algoritmo che identifichi la posizione dell'ultima occorrenza di 2 all'interno dell'array.

- Dare lo pseudocodice dell'algoritmo;
- Studiare la complessità dell'algoritmo proposto e discutere l'ottimalità.

Esempio:

input: $n = 7$, $M = [2, 2, 2, 3, 3, 3, 3]$

output: 2

Elaborazione Soluzione n.1: Una possibile prima soluzione potrebbe essere quella di scorrere tutti gli elementi, dal primo all'ultimo fintanto che non si trova un 3, e restituire la posizione precedente. Questa strategia non sfrutta alcuna informazione in nostro possesso, e pertanto può essere considerata una soluzione a forza bruta.

Pseudocodice:

Brute-force(Array M)

int $i = 0$

int $index = -1$

while $i \leq M.LENGTH()$ **do**

if $M[i] == 3$ **then**

$index = i - 1$

return: $index$

$i = i + 1$

Complessità: L'algoritmo proposto valuta ad una ad una tutte le posizioni dell'array e restituisce quando trova un 3 l'indice della posizione precedente.

Pertanto la soluzione in presenza di almeno un 3 esegue esattamente $\Theta(k)$ passi, se k è l'indice del primo 3. Tuttavia, il caso peggiore si incontra quando non esiste alcun 3, quando quindi l'algoritmo richiede un numero di passi lineari rispetto alla dimensione dell'input $\Theta(n)$, assunto n dimensione dell'array.

Ottimalità: l'algoritmo non è ottimo, perché il problema che dobbiamo affrontare può essere ridotto ad un problema di ricerca su di un array di elementi ordinati, pertanto possiamo evitare di valutare esattamente tutti gli elementi.

Elaborazione Soluzione n.2: Al fine di scrivere un algoritmo più veloce nel convergere alla soluzione possiamo sfruttare la ricerca binaria.

```

Last(Array  $M$ , int  $i$ , int  $j$ )
if  $j - i + 1 \leq 3$  then
     $t = i$ 
    while  $t < j$  do
         $t++$ 
        if  $M[t] == 2$  then
            return:  $t$ 
        else
            return:  $-1$ 
else
     $q = \lfloor \frac{i+j}{2} \rfloor$ 
    if  $M[q] == 2$  then
        return: Last( $M$ ,  $q$ ,  $j$ )
    if  $M[q] == 3$  then
        return: Last( $M$ ,  $i$ ,  $q - 1$ )

```

La ricerca binaria appena vista ha come caso base un array di al più 3 elementi. La sua complessità è $O(\log n)$.

Segue ora una versione della ricerca binaria che ha come caso base un array di dimensione al più 1.

```

Last-2(Array  $M$ , int  $i$ , int  $j$ )
if  $i == M.LENGTH()$  or  $j == 0$  then
    return:  $-1$ 
if  $i == j$  then
    return:  $i - 1$ 
 $q = \lfloor \frac{i+j}{2} \rfloor$ 
if  $M[q] == 2$  then
    return: Last-2( $M$ ,  $q$ ,  $j$ )
if  $M[q] == 3$  then
    return: Last-2( $M$ ,  $i$ ,  $q$ )

```

Funzionamento: L'idea alla base di LAST-2 è di ricorrere nella metà inferiore del vettore quando si incontra un 3, mentre di ricorrere sulla metà superiore quando si incontra un 2. Se i due indici (i , j) che tracciano la parte inferiore e superiore del sotto-array che si sta valutando coincidono, significa che ci siamo fermati sul primo 3 della sequenza e quindi dobbiamo ritornare la posizione precedente.

Viceversa, quando la i corrisponde alla lunghezza di M o la j corrisponde a 0, significa che ci siamo spostati esclusivamente verso destra o verso sinistra, come conseguenza di una sequenza di soli 2 o 3, e pertanto dobbiamo restituire -1 .

Dato che l'algoritmo dimezza ad ogni passo la dimensione dell'input, la complessità di LAST-2 sarà $O(\log n)$.

Ottimalità: Lo spazio delle soluzioni (ossia, i possibili valori ritornati dall'algoritmo) ha cardinalità $n + 1$ (dove $+1$ sta per indicare il caso che la sequenza non abbia nessun 2). L'albero delle decisioni ci dice che per scegliere fra $n + 1$ soluzioni sono necessarie almeno $\Omega(\log n)$ confronti. Pertanto gli algoritmi basati sulla ricerca binaria sono ottimi.

Esercizio 3.2.2 #Ottimalità

Dato un array A di dimensione n tale che $A[i] \in R$ e $1 \leq i \leq n$, trovare gli indici u e v tali che: $A[u] - A[v] = \max(A[i] - A[j])$ con $1 \leq i, j \leq n$.
Progettare e dare lo pseudocodice dell'algoritmo proposto, analizzare la complessità in tempo della soluzione proposta e discuterne l'ottimalità.

Elaborazione Soluzione n.1: Realizziamo un algoritmo a forza bruta che calcola tutte le combinazioni.

Complessità: Stiamo quindi confrontando n valori, uno a uno con gli altri n valori, per un totale di $\Theta(n^2)$ operazioni.

Ottimalità: La soluzione a forza bruta non è ottima. Il problema ha complessità intrinseca di $\Omega(n)$; infatti abbiamo un array in input di n elementi e dobbiamo guardare ogni elemento prima di poter concludere.

Elaborazione Soluzione n.2: Massimizzare $A[i] - A[j]$ è equivalente a fare la differenza del max e del minimo in A . Infatti, per ogni i, j , $A[i] - A[j] \leq A[i] - \min(A) \leq \max(A) - \min(A)$.

Pseudocodice:

Ricerca_Indici_Ottimale (Array $A[n]$)

int $index_max = 1$

int $index_min = 1$

for $i = 1$ **to** n **do**

if $A[i] > A[index_max]$ **then**

$index_max = i$

else

if $A[i] < A[index_min]$ **then**

$index_min = i$

return: ($index_max, index_min$)

Complessità: L'algoritmo costa $O(n)$ ed è ottimo.

Esercizio 3.2.3 #Ottimalità

Dato un array A che contiene interi ripetuti nel range $I = [1, 10]$, scrivere un algoritmo che usa $O(1)$ memoria aggiuntiva per trovare le occorrenze in A di ciascun intero di I .

Progettare l'algoritmo, darne lo pseudocodice, e discuterne la complessità.

Esempio:

Input: $A = 2, 2, 7, 3, 3, 5, 2$

Output: 0, 3, 2, 0, 1, 0, 1, 0, 0, 0

Elaborazione Soluzione: Scorriamo l'algoritmo dall'inizio alla fine contando le occorrenze dei 10 valori (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) che possono esservi contenuti. Questa soluzione ha costo spaziale $O(1)$, perché al massimo vengono create 10 variabili contatore, meglio un array di 10 celle, e costo temporale $O(n)$, perché scorre l'array A una sola volta senza eseguire operazioni "costose".

Pseudocodice: Supponiamo che gli array inizino da indice 0.

Min-Max(Array $A[n]$)

int $C[10] = \{0 \cdots 0\}$

for $i = 0, i \leq n, i++$ **do**

$C[A[i]]++$

return: C

Complessità: L'algoritmo ha costo temporale $O(n)$ dato dal ciclo **for** contenente l'operazione di somma. Il costo spaziale è $10 \in O(1)$ infatti viene allocato solo un array di dimensione 10.

Ottimalità: L'algoritmo è ottimo perché tutti i valori nell'array debbono essere analizzati.

Esercizio 3.2.4 #Ottimalità

Dato un array A contenente n interi, proporre un algoritmo per determinare se esiste almeno un intero duplicato in A .

Progettare e dare lo pseudocodice dell'algoritmo proposto.

Analizzare la complessità in tempo della soluzione proposta. Come cambia la complessità se l'array è ordinato? Discutere l'ottimalità.

Elaborazione Soluzione: Ordiniamo l'array e vediamo se vi sono valori consecutivi uguali leggendolo; questo ha costo $O(n \cdot \log n) + O(n) = O(n \cdot \log n)$.

Un'altra soluzione è quella di trovare il massimo ed il minimo, creare un array di tale dimensione e nelle celle scrivere per quante volte si ripetono i valori; questo ha costo $O(n) + (max - min)$.

Potremo combinare le due. Calcoliamo $(max - min)$, se è minore di $(n \cdot \log n)$ si va con la seconda soluzione e se maggiore con la prima.

Pseudocodice:

```

Find-Duplicate(Array  $A[n]$ )
int ( $max, min$ ) = FIND-MAX-AND-MIN( $A[n]$ )
if  $max - min > n \cdot \log n$  then
    MERGE-SORT( $A[n]$ )
    int  $i = 2$ 
    int  $ripetuto = A[1]$ 
    bool  $trovato = \text{FALSE}$ 
    while  $i \leq (n)$  and  $trovato == \text{FALSE}$  do
        if  $A[i] == ripetuto$  then
             $trovato = \text{TRUE}$ 
        else
             $ripetuto = A[i]$ 
         $i++$ 
    return:  $trovato$ 
else
    int  $C[max - min + 1] = \{0...0\}$  // Pieno di 0
    bool  $trovato = \text{FALSE}$ 
     $i = 1$ 
    while  $i \leq n$  and  $trovato == \text{FALSE}$  do
        if  $C[A[i] - min] == 0$  then
             $C[A[i] - min]++$ 
        else
             $trovato = \text{TRUE}$ 
    return:  $trovato$ 

```

L'operazione **FIND-MAX-AND-MIN**($A[n]$) ha costo $\Theta(n)$, scorre l'array dall'inizio alla fine e tiene traccia del valore minimo e massimo trovati.

Complessità: Nel caso peggiore, $(max - min) > \lceil n \cdot \log n \rceil$, abbiamo costo $O(n \cdot \log n)$. Poiché i numeri non sono ordinati questi devono essere controllati tutti per trovare la soluzione al problema, quindi abbiamo lower bound suggerito $\Omega(n)$ dalla dimensione dell'input.

Esercizio 3.2.5 #Ottimalità

Sia dato un insieme A di $2n$ interi, ciascuno compreso fra 1 e n (ossia, per ogni $1 \leq i \leq 2n$, $1 \leq A[i] \leq n$).

Determinare per ogni intero in $[1 : n]$, quanti sono gli interi in A minori di j .

Progettare e dare lo pseudocodice dell'algoritmo, discutere la complessità in tempo e l'ottimalità.

Elaborazione Soluzione: Una variante del **COUNTING-SORT**. Contiamo le occorrenze nell'array $C[1:n]$ e poi calcoliamo le somme prefisse.

Pseudocodice:**Find-Lower-Numbers**(Array $A[2n]$)**int** $C[n] = \{0 \cdots 0\}$ **for** $i = 1, i \leq 2n, i++$ **do** $C[A[i]]++$ **int** $P[n] = \{0 \cdots 0\}$ $P[1] = C[1]$ **for** $j = 2, j \leq n, j++$ **do** $P[j] = C[j] + P[j-1]$ **return** P

Per ottimizzare il secondo **for** si esegue in modo “furbo” la somma delle occorrenze dei valori $\leq j$ calcolati in $C[j]$.

$$P[j] = \sum_{t=1}^j C[t] = C[j] + \sum_{t=1}^{j-1} C[t] = C[j] + P[j-1]$$

Complessità: Abbiamo in input un array di dimensione $2n$ senza proprietà particolari, se non che i numeri al suo interno sono nel range $[1, n]$; ed un output di dimensione n , dato dalla proprietà. Il lower bound è $\Omega(n)$ per la dimensione dell’input, da cui l’algoritmo realizzato è ottimo.

Esercizio 3.2.6 #Ottimalità #TeoremaEsperto

Dato un array A di dimensione n che contiene caratteri maiuscoli seguiti da cifre comprese fra 0 e 9, trovare quanti caratteri sono memorizzati in A .

- Descrivere l’algoritmo a parole e dare lo pseudocodice
- Studiare la complessità in tempo della soluzione proposta
- Discutere l’ottimalità della soluzione proposta

Esempio di istanza: Array A di dimensione $n = 12$,

B	U	O	N	A	N	N	O	2	0	2	5
-----	-----	-----	-----	-----	-----	-----	-----	---	---	---	---

Output = 8

Elaborazione Soluzione: Cerchiamo in quale punto vi è l’ultimo carattere; supponiamo che ci sia almeno un carattere e almeno una cifra compresa fra 0 e 9. Applichiamo la Ricerca Binaria.

Sfruttando la struttura dell’array, se visitato un elemento è un carattere, tutti gli elementi che lo precedono sono caratteri e quindi l’ultimo carattere sarà l’elemento visitato o alla destra dell’elemento visitato; se l’elemento visitato è una cifra compresa fra 0 e 9, tutti gli elementi che lo seguono sono cifre comprese fra 0 e 9 e quindi l’ultimo carattere precede l’elemento visitato.

Pseudocodice:

```

Cerca_Caratteri(Array  $A$ , int  $s$ , int  $e$ )
if  $(e - s + 1) \leq 3$  then
    int  $i = s$ 
    while  $\text{IS\_CHAR}(A[i]) == \text{TRUE}$  and  $i < e$  do
         $i++$ 
    return:  $i$ 
else
    int  $q = \lfloor \frac{s+e}{2} \rfloor$ 
    if  $\text{IS\_CHAR}(A[q]) == \text{TRUE}$  then
         $\text{Cerca\_Caratteri}(A, q, e)$ 
    else
         $\text{Cerca\_Caratteri}(A, s, q - 1)$ 

```

Chiamata per il caso dell'esempio: $\text{CERCA_CARATTERI}(A, 1, 12)$.

Complessità: Detto $n = e - s + 1$, si ha:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 3 \\ T(\lfloor \frac{n}{2} \rfloor) + \Theta(1) & \text{if } n > 3 \end{cases}$$

Quindi, per il secondo caso del Teorema dell'Esperto: $T(n) = \Theta(\log n)$.

Ottimalità: Lo spazio delle soluzioni ha cardinalità n ; la complessità intrinseca del problema è $\Omega(\log n)$, che corrisponde alla minima altezza di un albero delle decisioni che sia capace di allocare n foglie, che corrispondono alle possibili soluzioni di questo algoritmo.

Esercizio 3.2.7 Dato un vettore A ordinato di n elementi interi distinti nel range $1 : n + 1$,

trovare il numero mancante. Si scriva lo pseudo-codice per trovare il numero mancante.

Si provi la correttezza del codice.

Si discuta la complessità in tempo e in spazio dell'algoritmo proposto.

Esercizio 3.2.8 Dato un vettore A ordinato di n elementi interi distinti nel range $1 : n - 1$

tranne uno che è ripetuto 2 volte, trovare il numero ripetuto. Si provi la correttezza del codice.

Si discuta la complessità in tempo e in spazio dell'algoritmo proposto.

Esercizio 3.2.9 Dato un vettore A di n interi compresi fra 0 e k^2 progettate e descrivete ad alto livello:

- un algoritmo che, nel caso pessimo, ordina A in tempo $O(n + k^2)$;
- un algoritmo che, nel caso pessimo, ordina A in tempo $O(n \log n)$.

Esercizio 3.2.10 Un array A di n , con $n \geq 3$, interi é detto *discontinuo* se esiste un indice

m , con $m \geq 1$ tale che $A[0] = \dots = A[m - 1] < A[m] = \dots = A[n - 1]$. Dato un array discontinuo, progettare e dare una realizzazione in pseudo-codice di un algoritmo che determini m in tempo $O(\log n)$. Discutere l'ottimalità della soluzione proposta.

Esercizio 3.2.11 Un array $A[1..n]$ contiene gli interi fra 0 e n , tranne uno. Ciascun intero è rappresentato in binario. Non è possibile leggere un intero completo di A , ma l'unica operazione di lettura possibile è leggere il bit j -esimo di $A[i]$, con $0 \leq j \leq \lfloor \log_2(n) \rfloor$ e $1 \leq i \leq n$. Tale operazione richiede tempo costante. Progettare un algoritmo *divide-et-impera* che in tempo $O(n)$ trova l'intero mancante.

(punti 2) Descrivere a parole la struttura dell'algoritmo di risoluzione

(punti 5) Dare una realizzazione dell'algoritmo in pseudocodice

(punti 5) Valutare la complessità in tempo dell'algoritmo spiegando il risultato indicato. Discutere l'ottimalità in tempo dell'algoritmo proposto.

Esercizio 3.2.12 Sia $f : \mathbb{N} \rightarrow \mathbb{R}$ una funzione monotona crescente, definita nell'intervallo $[a, b]$ tale che $f(a) < 0$ e $f(b) > 0$.

- Progettate un algoritmo *divide-et-impera* che determini $c \in [a, b]$ tale che $f(c) \leq 0 < f(c+1)$.
- Discutete l'ottimalità della soluzione proposta.

Esercizio 3.2.13 Progettare un algoritmo *divide-et-impera* per determinare l'elemento di rango 3 in un array $A = [1, \dots, n]$, contenente $n = 3^k$ interi. Dare una realizzazione dell'algoritmo in pseudocodice e giustificare la complessità della soluzione proposta.

Esercizio 3.2.14 Un array $A = [1, \dots, n]$ contiene n interi, tutti compresi fra 0 e n . Si assuma $n = 2^k$, con $k > 0$. Si assuma che l'array A sia ordinato in senso crescente. Descrivere un algoritmo che trova l'intero mancante in tempo $O(\log n)$. Dare una realizzazione dell'algoritmo in pseudocodice e giustificare la complessità della soluzione proposta.

Esercizio 3.2.15 Progettare un algoritmo per ordinare $O(n)$ numeri interi compresi nell'intervallo da 0 a $n^5 - 1$ in tempo $O(n)$. Descrivere a parole la soluzione proposta e giustificare la complessità.

Esercizio 3.2.16 Un vettore V di n interi distinti è detto *convesso-semplce* se esiste un indice j , con $1 \leq j \leq n$, tale che $V[1] = V[2] = \dots = V[j]$ e $V[i] < V[i+1]$ per $j \leq i < n$.

- Progettate un algoritmo che, dato un vettore convesso-semplce V di $n \geq 2$ interi, trovi l'indice j in tempo $O(\log n)$.
- Discutete l'ottimalità della soluzione proposta.

Esercizio 3.2.17 Progettare un algoritmo *divide-et-impera* per il calcolo del massimo di un vettore A non ordinato contenente n interi. Tale algoritmo divide A in tre parti e organizza ricorsivamente la ricerca. In particolare:

1. Descrivere a parole la struttura dell'algoritmo di risoluzione
2. Dare una realizzazione dell'algoritmo in pseudocodice
3. Valutare la complessità in tempo dell'algoritmo spiegando il risultato indicato. Discutere l'ottimalità in tempo dell'algoritmo proposto.

Esercizio 3.2.18 Consideriamo il seguente problema: Dati n punti $x_1, x_2, \dots, x_n$ su una linea ordinati, determinare la coppia di punti x_i e x_j la cui distanza $|x_i - x_j|$ è minima.

1. Descrivere a parole la struttura di un algoritmo **divide-et-impera** per risolvere tale problema.
2. Darne una realizzazione dell'algoritmo in pseudocodice
3. Discutere l'equazione di ricorrenza che descrive la complessità in tempo della soluzione proposta e trovarne un limite superiore.

Capitolo 4

Heaps

4.1 Problemi su Max-Heaps

Esercizio 4.1.1 #MaxHeap

Sia H un max-heap di dimensione n e sia $H[k]$ la chiave memorizzata in posizione k con $1 \leq k \leq n$ da decrementare di una unità. Si proponga un algoritmo che decrementi $H[k]$ e restituisca H modificato (Si noti, dopo la modifica, lo heap deve preservare le proprie caratteristiche di max-heap). Dare lo pseudocodice dell'algoritmo. Studiare la complessità in tempo della soluzione proposta.

Elaborazione Soluzione: Poiché $H[k]$ decresce, la nuova chiave può solo scendere nel max-heap, inoltre è la sola chiave che può violare le proprietà dello heap, quindi applichiamo la funzione MAXHEAPFY affinché ricollochi correttamente l'elemento in $H[k]$, così che lo scambi con uno più in basso.

```
DecrementaPosizione(Array  $H[n]$ , int  $k$ )
//  $H$  è l'heap e  $k$  la posizione in cui vi è l'elemento da decrementare
//  $n$  è la dimensione del heap
 $H[k] = H[k] - 1$ 
MAXHEAPFY( $H, k$ ) // Funzione che mantiene la proprietà dello heap in posizione  $k$ 
Per completezza riportiamo la funzione MAXHEAPFY:
maxHeapfy(Array  $H[n]$ , int  $k$ )
int  $max = k$ 
left =  $2k$  // Il figlio sinistro è in questa posizione
right =  $2k + 1$  // Il figlio destro è in questa posizione
if left <  $n$  then
    if  $H[k] < H[left]$  then
         $max = left$ 
    else
         $max = k$ 
if right <  $n$  then
    if  $H[max] < H[right]$  then
         $max = right$ 
//  $max$  contiene l'indice del nodo in  $k, 2k, 2k + 1$  con chiave maggiore
if  $max \neq k$  then
    SCAMBIA( $H[max], H[k]$ )
    maxHeapfy( $H, max$ )
```

Complessità: MAXHEAPFY ha complessità $O(\log n)$, ma, poiché lavoriamo su un sub-heap radicato in k , il costo sarà $O(\log n - \log k) = O(\log \frac{n}{k})$.

Esercizio 4.1.2 #MaxHeap #Ottimalità

Dato un max-heap H di dimensione n , progettare un algoritmo che cerca il minimo in H . Progettare e dare lo pseudocodice dell'algoritmo proposto. Studiare la complessità dell'algoritmo proposto. Discutere l'ottimalità della soluzione proposta.

Elaborazione Soluzione: Sappiamo che in un max-heap il valore minimo è in una delle foglie. Sappiamo che un heap per via della rigidità della sua struttura può essere visto come un array; supponiamo che sia quindi memorizzato in tale struttura dati. Sappiamo anche che in un albero al massimo metà dei nodi possono essere foglie; essendo un max-heap allora il minimo sarà, nell'array che lo memorizza, tra la posizione $\lfloor \frac{n}{2} \rfloor + 1$ e la posizione n . Quindi basta fare una ricerca del minimo in quella porzione dell'array.

Esempio: Heap di 7 elementi con le foglie in posizione 4, 5, 6, 7; il minimo può stare in una qualsiasi foglia.

10, 4, 6, 1, 3, 2, 5

10, 4, 6, 3, 1, 2, 5

10, 4, 6, 3, 2, 1, 5

10, 4, 6, 3, 2, 5, 1

Quindi l'algoritmo pensato è ottimo.

Pseudocodice:

Find-Min(Array $H[n]$)

int $start = \lfloor \frac{n}{2} \rfloor + 1$

int $min = H[start]$

for $i = start + 1, i \leq n, i++$ **do**

if $H[i] < min$ **then**
 $min = H[i]$

return: min

Complessità ed Ottimalità: Questa soluzione ha costo $\lfloor \frac{n}{2} \rfloor + 1 \in O(n)$. Per stimare la complessità intrinseca del problema notiamo che il problema si riduce a cercare il minimo di un array (senza alcun ordinamento) con $\frac{n}{2}$ elementi. Quindi abbiamo il lower bound è $\Omega(n)$. Pertanto, l'algoritmo proposto è ottimo.

Esercizio 4.1.3 #MaxHeap

Dato un max-heap H i cui nodi contengono interi, ed un intero $1 \leq k \leq |H|$ proporre un algoritmo per raddoppiare il valore di $H[k]$ e restituire H modificato. Analizzare la complessità in tempo della soluzione proposta.

N.B. H deve soddisfare le proprietà del max-heap dopo le modifiche.

Elaborazione Soluzione: Essendo in un max-heap, il nuovo valore di $H[k]$ può solo risalire, fino al più alla radice.

Pseudocodice:

maxHeapfy(Array $H[n]$, int k)

$H[k] = 2 \cdot H[k]$

int $padre = \lfloor \frac{k}{2} \rfloor$

int $figlio = k$

while $padre \geq 1$ **and** $H[padre] < H[figlio]$ **do**

 SCAMBIA($H[padre]$, $A[figlio]$)

$figlio = padre$

$padre = \lfloor \frac{padre}{2} \rfloor$

Complessità: La complessità è $O(\log(k))$ ossia la distanza del nodo k dalla radice di H . La posizione della nuova chiave $H[k]$ si può determinare in tempo $O(\log \log k)$, ma la sua inserzione nell'opportuna posizione richiede di spostare le chiavi più piccole verso il basso. Quindi la complessità intrinseca del problema è $\Omega(\log k)$.

Esercizio 4.1.4 #MaxHeap #Correttezza

Dato un vettore A di n elementi, ordinato in senso crescente, si trasformi A in modo che dopo la trasformazione esso rappresenti un max-heap. Si scriva lo pseudo-codice per tale trasformazione. Si provi la correttezza del codice.
Si discuta la complessità in tempo e in spazio dell'algoritmo proposto.

Elaborazione Soluzione: Un array ordinato in senso decrescente è un max-heap (non vale il contrario, i max-heap non sono sempre array ordinati in senso decrescente), quindi basta capovolgere l'array.

Correttezza: Poiché ogni array ha la forma di uno heap e un array ordinato in senso decrescente soddisfa la max-heap property, la soluzione è corretta.

Pseudocodice:

Array-Rotation(Array $A[n]$)

int $t = 0$ // Variabile temporanea

for $i = 1, i \leq \lceil \frac{n}{2} \rceil, i++$ **do**

```

     $t = A[i]$ 
     $A[i] = A[n + 1 - i]$ 
     $A[n + 1 - i] = t$ 

```

Complessità: L'algoritmo realizzato ha costo spaziale $O(1)$, perché non viene allocata nuova memoria; si esegue il capovolgimento direttamente sull'array originale. Il costo temporale è di $O(n)$ perché per creare uno heap di dimensione n tutti gli elementi debbono essere analizzati.

Esercizio 4.1.5 Si consideri un *MAX heap quaternario* di n nodi e altezza h .

1. Descrivere le proprietà di un MAX heap quaternario.
2. Descrivere un'implementazione efficiente dello heap quaternario tramite un vettore A , indicando le formule per calcolare la posizione del padre, del primo, del secondo, del terzo e del quarto figlio di un nodo i . Si assuma la radice dello heap memorizzata in posizione 1 di A .
3. Dare una realizzazione in pseudocodice dell'operazione *Insert-Key*(A, k) per lo heap quaternario. Analizzare la complessità della soluzione proposta.
4. Dare una realizzazione in pseudocodice dell'operazione *Max-Heapify*(A, i) per lo heap quaternario. (*Max-Heapify*(A, i) ristabilisce le proprietà dello heap dopo che $A[i]$ è stato decrementato. I sottoalberi radicati nei figli di i non violano le proprietà dello heap). Analizzare la complessità della soluzione proposta.

Capitolo 5

Statistica d'Ordine

5.1 Ricerca di Elementi per Rango

Esercizio 5.1.1 #Ottimalità #Rango

Dato un array $A[n]$ di n elementi, trovare in esso l'elemento di rango i .
Fornire pseudocodice dell'algoritmo e delle procedure utilizzate.
Discutere la complessità della soluzione proposta e discuterne l'ottimalità.

Elaborazione Soluzione n.1:

Dato un insieme di elementi n , i -esima Statistica d'Ordine è l'elemento di rango (posizione) i nell'insieme ordinato, per convenzione, in senso crescente. Per cui, i -esima statistica d'ordine è l' i -esimo elemento più piccolo.

Es: Dato un array $\{4,7,2,6,10\}$, lo ordiniamo in verso **crescente** ottenendo $\{2,4,6,7,10\}$ dove 2 è l'elemento di rango 1, 4 quello di rango 2, 6 quello di rango 3, ecc...

Poiché cerchiamo quell'elemento che, nell'array ordinato, sarebbe in posizione i ; possiamo semplicemente ordinare l'array e ritornare l'elemento in posizione i . Tuttavia, in questo modo l'algoritmo costa $O(n \log n)$ (ordinamento con MERGE-SORT).

La complessità intrinseca del problema è però $\Omega(n)$, perché non abbiamo proprietà dell'array e quindi dobbiamo esaminare ogni suo elemento.

Una soluzione migliore è quella di realizzare una funzione, qui ricorsiva, per trovare il rango di un elemento casuale nell'array, detto pivot, per poi richiamare la funzione solo sul sotto-array che contiene il rango i .

Se siamo fortunati, quindi il pivot divide in due parti uguali l'array (ossia ha rango $n/2$), avremo un algoritmo che nel ramo asintotico ha complessità $T(n) = T(n/2) + \Theta(n)$, che per il terzo caso del Master Theorem è $T(n) \in \Theta(n)$, e quindi ottimo.

Pseudocodice:

Nel seguente algoritmo s ed e sono l'indice di inizio e di fine della porzione dell'array esaminato e i è la statistica d'ordine cercata.

Si assume $s \leq i \leq e$.

```

Selection(Array  $A[s \cdots e]$ , int  $s$ , int  $e$ , int  $i$  )
if  $s == e$  then
    return:  $A[e]$ 
else
     $P = \text{RANDOM}(s, e)$  // Indice a caso tra  $s$  ed  $e$  che individua il Pivot
     $q = \text{PARTITION}(A, s, e, P)$  // Rango del Pivot attorno cui abbiamo partizionato
    l'array
     $k = q - s + 1$  // Il numero di elementi nel sotto-array  $A[s \cdots q]$ 
    if  $i == k$  then
        return:  $A[q]$ 
        // Il valore del Pivot è l' $i$ -esima statistica d'ordine
    else
        if  $i < k$  then
            return: Selection( $A, s, q - 1, i$ )
            // Cerchiamo nei valori a sinistra del Pivot
        else
            return: Selection( $A, q + 1, e, i - k$ )
            // Cerchiamo nei valori a destra del Pivot

Partition(Array  $A[s \cdots e]$ , int  $s$ , int  $e$ , int  $j$ )
// La funzione trova il rango dell'ultimo elemento dell'array
SCAMBIA( $A[j], A[e]$ ) // Metto il  $j$ -esimo elemento nell'ultima posizione
 $x = A[e]$  // Memorizziamo il Pivot messo all'ultima posizione
 $y = s - 1$  // Indice dell'ultimo elemento trovato minore del pivot
for  $k = s$  to  $e - 1$  do
    if  $A[k] \leq x$  then
        // Se l'elemento osservato è  $\leq$  del Pivot, è spostato tra gli elementi minori o
        uguali al pivot
         $y = y + 1$ 
        // Scambiamo l'elemento osservato con quello nell'indice temporaneo del Pivot
        SCAMBIA( $A[y], A[k]$ )
// Gli elementi da  $s$  a  $y$  sono  $\leq x$ , gli elementi da  $y + 1$  a  $e - 1$  sono  $> x$ 
 $y = y + 1$  // Mettiamo il Pivot nell'indice che corrisponde al suo rango
SCAMBIA( $A[y], A[e]$ ) // Ritorniamo  $y$ , il Rango del Pivot
return:  $y$ 

```

Complessità: Il tempo di esecuzione nel caso peggiore del nostro algoritmo è di $O(n^2)$. Infatti se il rango del pivot è nell'intorno costante del minimo o del massimo, l'equazione di ricorrenza diventa $T(n) = \max\{T(c), T(n-c-1)\} + O(n)$, ove c è una costante, la cui soluzione è $O(n^2)$.

Elaborazione Soluzione n.2: In alternativa al selezionare casualmente il Pivot con la `RANDOM()`, possiamo fare in modo di garantire che il pivot scelto sia tale da bilanciare le chiamate ricorsive destra e sinistra; così da avere soluzione ottima anche nel caso peggiore.

Per farlo usiamo la procedura `MEDIANAMEDIANE()` qui descritta a parole:

```

MedianaMediane(Array  $A[s \cdots e]$ , int  $s$ , int  $e$ )
Creare  $n/5$  sotto-array di 5 elementi // Costo  $O(n)$ 
Eseguire un algoritmo di ordinamento su ciascuno di essi // Costo  $O(n)$ 
Selezionarne le mediane di ciascuno, per metterle nell'array  $M$  // Costo  $O(n)$ 
return SELECTION( $M, 0, \#M - 1, \lfloor \frac{\#M}{2} \rfloor$ ) // Costo  $T(n/5)$ 

```

Funzionamento: Si divide la sequenza in gruppi di 5 elementi; nell'ultimo potremo averne di meno, ma è, ai fini della complessità, irrilevante.

Si ordina ogni gruppo e si seleziona il suo valore centrale di rango 3, quindi la Mediana del

gruppo; creando un vettore delle mediane.

Infine si richiama ricorsivamente la procedura SELECTION sul vettore delle mediane per trovare la mediana delle $n/5$ mediane.

Es:

Cerchiamo la mediana nell'array:

4, 2, 8, 7, 6, 1, 5, 9, 7, 3, 10, 11, 15, 13, 14, 18, 1, 22, 20, 21, 9, 3, 6, 11, 10

Dividiamo l'array in sotto-
array di 5 elementi:

4 2 8 7 6
1 5 9 7 3
10 11 15 13 14
18 1 22 20 21
9 3 6 11 10

Ordiniamo ciascuno dei
sotto-array:

2 4 6 7 8
1 3 5 7 9
10 11 13 14 15
1 18 20 21 22
3 6 9 10 11

Ordiniamo verticalmente la
colonna mediana:

2 4 6 7 8
1 3 5 7 9
3 6 9 10 11
10 11 13 14 15
1 18 20 21 22

La nostra mediana delle mediane è il valore centrale: 9

Ottimalità: Utilizzando questa procedura l'elemento trovato come pivot è sufficientemente centrale da garantirci un certo equilibrio tra le chiamate destra e sinistra della funzione ricorsiva; garantendoci una soluzione ottima.

Infatti, il rango della mediana delle mediane nell'array cade nell'intorno di $\frac{n}{2}$; il numero di elementi maggiori della mediana delle mediane è al più: $(\frac{n}{5} \cdot \frac{1}{2} - 2) \cdot 3 = \frac{3n}{10} - 6$. Analogamente il numero di elementi minori della mediana delle mediane è al meno: $\frac{n}{5} \cdot \frac{1}{2} \cdot 3 = \frac{3n}{10}$.

A seconda del valore della statistica d'ordine cercata si avrà chiamata ricorsiva o a destra, ossia sugli elementi con rango maggiore del rango della mediana delle mediane, che sono al più $n - (\frac{3n}{10}) = \frac{7n}{10}$; o a sinistra, ossia sugli elementi con rango minore del rango della mediana delle mediane che sono al più $n - \frac{3n}{10} + 6 = \frac{7n}{10} + 6$.

Quindi il costo complessivo dell'algoritmo per il calcolo della mediana delle mediane è:

$$T(n) = \Theta(n) + T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right)$$

Si dimostra per induzione essere in $\Theta(n)$.

Esercizio 5.1.2 #Ottimalità #Rango

Dati due insiemi S e T di n numeri distinti e dato un intero positivo $k < n$, determinare gli elementi di rango minore o uguale a k in $S \circ T$, dove $\circ$ indica la giustapposizione di tutti gli elementi di S con quelli di T (i valori presenti sia in S che in T saranno replicati in $S \circ T$).

- Dare lo pseudocodice dell'algoritmo (Non è necessario scrivere lo pseudocodice delle procedure studiate a lezione che verranno eventualmente invocate)
- Discutere la complessità dell'algoritmo proposto e discutere la sua ottimalità (l'esercizio sarà valutato con il punteggio massimo se la soluzione proposta ha complessità ottima)
- Come cambia la complessità se S e T sono due insiemi ordinati?

Elaborazione Soluzione: Uniamo i due array S e T in un nuovo array U ed invochiamo SELECTION su U con rango k . Selection ritorna P l'elemento in U con rango k . Infine invochiamo PARTITION rispetto al valore P . Gli elementi alla sinistra di P al termine della procedura saranno gli elementi di rango minore o uguale a k in $S \cup T$. La procedura ha costo

$O(|S| \cup |T|) = O(n)$.

Pseudocodice:

Premesso che: La SELECTION viene modificata per tornare non la k -esima statistica d'ordine, ma l'indice nell'array dove si trova la k -esima statistica d'ordine.

rank-lessEqual-k(Array $S[n]$, Array $T[n]$, int k)

$U = S \cup T$ // array di dimensione $2n$

$q = \text{SELECTION}(U, 1, 2n, k)$ // q è l'indice dell'elemento di rango k nell'array U

$k = \text{PARTITION}(U, 1, 2n, q)$ // fa la partizione sull'array in-place e ritorna l'indice di q in U

return $U[1 \dots k]$

Complessità: Creare il nuovo array U costa $2n$, perché unisce due array di dimensione n , sia la SELECTION che la PARTITION costano $2n$, perché lavorano su U di tale dimensione e la stampa/ritorno del risultato costa k ; quindi il costo finale è $2n + 2n + 2n + k \in O(n)$ perché $k < n$.

Elaborazione Soluzione 2: Si osservi che l'elemento di rango k in $S \circ T$, ha rango minore o uguale a k sia in S che in T . Se $n \gg k$ (es. $n = 10^8, k = 15$), è vantaggioso lavorare il più possibile su array più piccoli. Pertanto, una soluzione efficiente se $n \gg k$ estrae i k elementi più piccoli di S e di T , ordina i due sotto-array di k elementi e ne fa il merge troncandolo quando trova i k elementi più piccoli.

rank-lessEqual-k(Array $S[n]$, Array $T[n]$, int k)

$q = \text{SELECTION}(S, 1, n, k)$ // q è l'indice dell'elemento di rango k nell'array S

$S'[1 : k] = \text{PARTITION}(S, 1, n, S[q])$

$q2 = \text{SELECTION}(T, 1, n, k)$ // $q2$ è l'indice dell'elemento di rango k nell'array T

$T'[1 : k] = \text{PARTITION}(T, 1, n, T[q2])$

$\text{MERGESORT}(S')$

$\text{MERGESORT}(T')$

$U = \text{MERGE}(S', T', k)$

return $U[1 \dots k]$

Complessità: $O(n + k \log k + k) \in O(n)$. Asintoticamente è la stessa complessità, ma la costante nascosta nell'ordine di grandezza della seconda soluzione è assai più piccola di quella nella prima soluzione.

Nel caso in cui gli array S e T siano **ordinati** possiamo semplicemente impostare 3 contatori, uno che sarà un **for** da 1 a k e due per scorrere gli array, e leggere gli array prendendo ad ogni *tick* del contatore la cifra più piccola che leggiamo tra le due, per poi aumentare il contatore dell'array da cui si è preso il valore; quindi il costo è $O(k)$. In questo caso, la soluzione è una versione modificata dell'algoritmo MERGE.

Pseudocodice:

rank-lessEqual-k-ordinati(Array $S[n]$, Array $T[n]$, int k)

int $i = 1$

int $j = 1$

int $R[1 \dots k] = \{\}$ // Array con il risultato

for $c = 1, c \leq k, c++$ **do**

if $S[i] \geq T[j]$ **then**

$R(c) = T[j]$

$j++$

else

$R(c) = S[i]$

$i++$

return: $R[1 \dots k]$

Complessità: Il costo computazionale è $O(k)$. L'algoritmo è ottimo se si richiede di restituire un array che contiene le k chiavi più piccole.

Bibliografia

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms, 4th Edition*. MIT Press, 2022.

Indice delle Etichette

AlberoRicorrenze, 10, 15, 16, 19

ConfrontoFunzioni, 4, 5

Correttezza, 40

CountingSort, 24, 25

Induzione, 10, 11, 13

IpotesiOrdineInferiore, 11

Iterazione, 6, 8, 19

MaxHeap, 39, 40

MergeSort, 27

OrdineGrandezza, 1, 3–5

Ottimalità, 21, 23–25, 29, 31–35, 39, 43, 45

Rango, 43, 45

RicercaBinaria, 23, 29, 31

Ricorsione, 15, 16, 19

TeoremaEsperto, 11, 13, 35

Quest’opera è distribuita con licenza Creative Commons “Attribuzione
– Non commerciale 4.0 Internazionale”.

